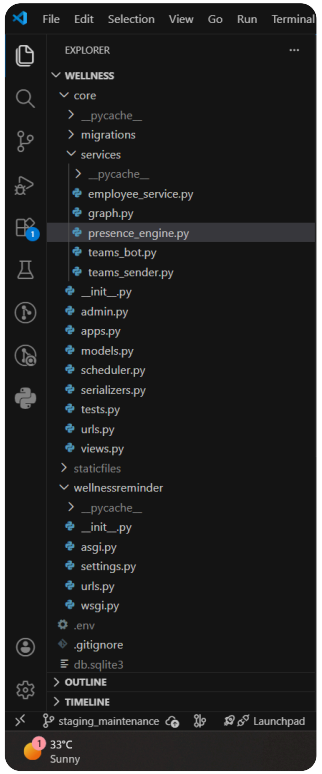



```
from core.models import Employee
from core.services.graph import get_user_profile


def get_or_create_employee_from_teams(
    aad_object_id,
    teams_user_id,
    conversation_id,
    tenant_id,
    service_url,
```

```
    fallback_name=None,
):
    """
    Auto-create employee when user first messages bot.
    If employee exists, update Teams conversation details.
    """
    if not aad_object_id:
        raise Exception("Missing aad_object_id from Teams activity")

    employee =
Employee.objects.filter(azure_user_id=aad_object_id).first()

    if employee:
        # Update Teams conversation details every time
        employee.teams_user_id = teams_user_id
        employee.teams_conversation_id = conversation_id
        employee.teams_tenant_id = tenant_id
        employee.teams_service_url = service_url
        employee.save(update_fields=[
            "teams_user_id",
            "teams_conversation_id",
            "teams_tenant_id",
            "teams_service_url",
        ])
        return employee, False

    # Not found -> fetch profile from Graph
    profile = get_user_profile(aad_object_id)
```

```
    display_name = profile.get("displayName") or fallback_name or
"Unknown User"
    email = (
        profile.get("mail")
        or profile.get("userPrincipalName")
        or f"{aad_object_id}@example.com"
    )

    employee = Employee.objects.create(
        azure_user_id=aad_object_id,
        display_name=display_name,
        email=email,
        teams_user_id=teams_user_id,
        teams_conversation_id=conversation_id,
        teams_tenant_id=tenant_id,
        teams_service_url=service_url,
        is_active=True,
        reminder_enabled=True,
        reminder_interval_minutes=60,
    )

    return employee, True
employee_service.py
import requests
import msal
from django.conf import settings

AUTHORITY =
f"https://login.microsoftonline.com/{settings.GRAPH_TENANT_ID}"
```

```
SCOPE = ["https://graph.microsoft.com/.default"]

_app = msal.ConfidentialClientApplication(
    settings.GRAPH_CLIENT_ID,
    authority=AUTHORITY,
    client_credential=settings.GRAPH_CLIENT_SECRET,
)


def get_access_token():
    """
    Acquire app-only access token using MSAL.
    MSAL handles token caching automatically.
    """
    result = _app.acquire_token_for_client(scopes=SCOPE)

    if "access_token" in result:
        return result["access_token"]

    raise Exception(f"Token error: {result}")


def get_user_presence(user_id):
    """
    Get Teams presence for a user using Azure AD Object ID.
    """
    token = get_access_token()

    url = f"https://graph.microsoft.com/v1.0/users/{user_id}/presence"
```

```
    headers = {
        "Authorization": f"Bearer {token}",
        "Content-Type": "application/json",
    }

    response = requests.get(url, headers=headers)

    if response.status_code == 200:
        return response.json()

    raise Exception(f"Graph Presence Error {response.status_code}:
{response.text}")


def get_user_profile(user_id):
    """
    Fetch user profile from Microsoft Graph using Azure AD Object ID.
    Useful for auto-registering employee on first bot message.
    """
    token = get_access_token()

    url = (
        f"https://graph.microsoft.com/v1.0/users/{user_id}"
        f"?$select=id,displayName,mail,userPrincipalName"
    )

    headers = {
        "Authorization": f"Bearer {token}",
```

```
        "Content-Type": "application/json",
    }

    response = requests.get(url, headers=headers)

    if response.status_code == 200:
        return response.json()

    raise Exception(f"Graph Profile Error {response.status_code}:
{response.text}")
```

graph.py

```
import math

from asgiref.sync import async_to_sync
from django.utils import timezone

from core.models import Employee, PresenceLog, ActivityTracker,
ReminderLog, GoodMorningLog
from core.services.graph import get_user_presence
from core.services.teams_sender import send_proactive_message


WORKING_STATUSES = [
    "Available",
    "Busy",
    "InACall",
    "InAMeeting",
]
```

```
GOOD_MORNING_ALLOWED_STATUSES = [
    "Available",
    "Busy",
    "InACall",
    "InAMeeting",
]

CHECK_INTERVAL_MINUTES = 2

WELLNESS_REMINDER_MESSAGES = [
    "✨ Small break, big benefit — take 2 minutes to move and recharge.",
    "🙆 Stretch reminder! Roll your neck and shoulders gently to reduce stiffness.",
    "🖥️ Screen break time — rest your eyes and blink a few times slowly.",
    "🧍 Quick posture check! Sit tall or stand up and stretch for a moment.",
    "🌬️ Take a deep breath — a short breathing break can improve focus.",
    "💧 Hydration check! Take a sip of water and relax your shoulders.",
    "🧘 Short break time! Stand up, stretch, and refresh your body for 2 minutes.",
    "👀 Give your eyes a break — look away from the screen for a few moments.",
    "🚶 Time to move a little! A short walk can help reset your focus.",
    "🌿 Pause for a moment — breathe deeply and reset your energy.",
    "💺 Sitting too long? Stand up and loosen your back and legs for a minute.",
```

```
    "🚰 Wellness check! Drink water and take a mindful pause before continuing.",
    "⏰ Time for a quick wellness break! Please drink some water and stretch a little.",
    "🚰 Friendly reminder: hydrate, stretch, and return refreshed.",
]

GOOD_MORNING_MESSAGES = [
    "🌞 Good morning! Hope you start your day with energy, focus, and a healthy routine.",
    "🌷 Good morning! Wishing you a calm mind, good posture, and a productive day ahead.",
    "💧 Good morning! Stay hydrated, take mindful breaks, and have a wonderful workday.",
    "🌿 Good morning! A fresh start for a fresh day — work well and remember to take care of yourself.",
    "✨ Good morning! Small healthy habits today can make a big difference — have a great day ahead.",
    "🌞 Good morning! Wishing you a productive and healthy workday. I'll gently remind you to take wellness breaks during the day.",
    "🧘 Good morning! Begin your day with focus and balance. I'll be here to remind you to take short wellness breaks.",
    "🚶 Good morning! Wishing you a positive day filled with productivity and healthy movement.",
    "🌸 Good morning! Stay active, stretch often, and make today a great one.",
    "🌈 Good morning! Hope your day is productive, peaceful, and full of positive energy.",
    "💼 Good morning! Let's make today productive and healthy —
```

```
don’t forget to stretch and hydrate.",
    "😊 Good morning! Wishing you a smooth workday with good focus, good health, and good energy.",
]


def is_within_working_hours():
    """
    Monday-Friday, 8:00 AM to 7:00 PM (Asia/Kolkata)
    """
    now = timezone.localtime()
    weekday = now.weekday()  # Monday=0, Sunday=6
    current_hour = now.hour

    is_weekday = weekday < 5
    within_hours = 8 <= current_hour < 19

    return is_weekday and within_hours


def get_next_reminder_message(employee):
    """
    Return a reminder message that has not yet been used today for this employee.
    If all messages have been used today, fallback to first message.
    """
    today = timezone.localdate()

    used_messages_today = set(
```

```
        ReminderLog.objects.filter(
            employee=employee,
            sent_at__date=today
        ).values_list("message", flat=True)
    )

    for message in WELLNESS_REMINDER_MESSAGES:
        if message not in used_messages_today:
            return message

    # Fallback if all messages used in same day (unlikely but safe)
    return WELLNESS_REMINDER_MESSAGES[0]

def get_next_good_morning_message(employee):
    """
    Return a Good Morning message for this employee.

    Rules:
    - Only one Good Morning per employee per day
    - Since old GoodMorningLogs are deleted daily, do NOT rely on
history
    - Rotate message using today's date + employee id
    """
    today = timezone.localdate()

    # Prevent duplicate Good Morning on same day
    already_sent_today = GoodMorningLog.objects.filter(
        employee=employee,
        sent_at__date=today
```

```
    ).exists()

    if already_sent_today:
        return None

    # Rotate messages daily using date + employee id
    # Works for UUID primary keys also
    day_number = today.toordinal()

    if employee.id:
        employee_seed = employee.id.int if hasattr(employee.id, "int") else
int(employee.id)
    else:
        employee_seed = 0

    index = (day_number + employee_seed) %
len(GOOD_MORNING_MESSAGES)
    return GOOD_MORNING_MESSAGES[index]

def has_teams_chat_initialized(employee):
    """
    Check whether employee has opened the bot at least once and
conversation details exist.
    """
    return (
        bool(employee.teams_user_id)
        and bool(employee.teams_conversation_id)
        and bool(employee.teams_service_url)
    )
```

```
def fetch_employee_presence(employee, save_log=True):
    """
    Fetch employee presence safely.
    Returns status string or None if fetch fails.

    If save_log=True, also stores PresenceLog.
    """
    try:
        presence_data = get_user_presence(employee.azure_user_id)
        status = presence_data.get("availability")

        print(f"{employee.display_name} status: {status}")

        if save_log:
            PresenceLog.objects.create(
                employee=employee,
                status=status or "Unknown"
            )

        return status

    except Exception as e:
        print(f"Presence fetch failed for {employee.display_name}: {e}")
        return None


def check_employee_presence():
```

```
    """
    Runs every 2 minutes during working hours.
    Sends reminder after continuous working duration reaches employee
threshold.
    """
    if not is_within_working_hours():
        print("Outside working hours. Skipping presence check.")
        return

    employees = Employee.objects.filter(
        is_active=True,
        reminder_enabled=True,
    )

    for employee in employees:
        # User must have initialized chat at least once
        if not has_teams_chat_initialized(employee):
            print(f"Skipping {employee.display_name}: no Teams
conversation details yet")
            continue

        status = fetch_employee_presence(employee, save_log=True)

        if status is None:
            continue

        tracker, _ =
ActivityTracker.objects.get_or_create(employee=employee)
```

```
        if status in WORKING_STATUSES:
            tracker.consecutive_available_count += 1
            print(f"Working detected -> {employee.display_name}:
{tracker.consecutive_available_count}")
        else:
            print(f"Resetting counter for {employee.display_name}")
            tracker.consecutive_available_count = 0

        # Example:
        # 60 mins / 2 mins = 30 checks
        # Use ceil so reminders are never sent earlier than configured
interval
        required_checks = max(
            1,
            math.ceil(employee.reminder_interval_minutes /
CHECK_INTERVAL_MINUTES)
        )

        if tracker.consecutive_available_count >= required_checks:
            reminder_message = get_next_reminder_message(employee)

            print(f"Reminder triggered for {employee.display_name}")
            print(f"Reminder message: {reminder_message}")

            try:
                async_to_sync(send_proactive_message)(employee,
reminder_message)
                print(f"Reminder sent to Teams for
{employee.display_name}")
```

```
            # Create log only after successful send
            ReminderLog.objects.create(
                employee=employee,
                message=reminder_message
            )
            print(f"Reminder log created for {employee.display_name}")

            # Reset only after successful send
            tracker.consecutive_available_count = 0

        except Exception as e:
            print(f"Failed to send Teams reminder to
{employee.display_name}: {e}")

    tracker.last_status = status
    tracker.save()


def send_good_morning_messages():
    """
    Sends Good Morning message only to active employees who:
    - have initialized Teams chat
    - are currently online/working (not offline/away)
    - have NOT already received Good Morning today

    Runs at scheduled time Mon-Fri.
    """
    now = timezone.localtime()
```

```
    weekday = now.weekday()

    if weekday >= 5:
        print("Weekend. Skipping good morning messages.")
        return

    employees = Employee.objects.filter(
        is_active=True,
        reminder_enabled=True,
    )

    for employee in employees:
        if not has_teams_chat_initialized(employee):
            print(f"Skipping good morning for {employee.display_name}: no
Teams chat initialized")
            continue

        # Prevent duplicate Good Morning in same day
        good_morning_message =
get_next_good_morning_message(employee)
        if good_morning_message is None:
            print(f"Skipping good morning for {employee.display_name}:
already sent today")
            continue

        # Fetch live presence before sending Good Morning
        status = fetch_employee_presence(employee, save_log=True)

        if status is None:
```

```
            print(f"Skipping good morning for {employee.display_name}: 
presence unavailable")
            continue

        if status not in GOOD_MORNING_ALLOWED_STATUSES:
            print(
                f"Skipping good morning for {employee.display_name}: "
                f"status is {status} (not eligible)"
            )
            continue

        try:
            async_to_sync(send_proactive_message)(employee, 
good_morning_message)
            print(f"Good morning message sent to 
{employee.display_name}")
            print(f"Good morning message: {good_morning_message}")

            # Create log only after successful send
            GoodMorningLog.objects.create(
                employee=employee,
                message=good_morning_message
            )
            print(f"Good morning log created for 
{employee.display_name}")

        except Exception as e:
            print(f"Failed to send good morning to 
{employee.display_name}: {e}")
```

```
def reset_daily_trackers():
    """
    Reset all activity counters before workday starts.
    Also delete all old PresenceLogs and GoodMorningLogs
    so only today's logs remain.
    Runs at 7:55 AM Mon-Fri.
    """
    today = timezone.localdate()

    # Delete old presence logs
    deleted_presence_count, _ = PresenceLog.objects.exclude(
        checked_at__date=today
    ).delete()

    # Delete old good morning logs
    deleted_good_morning_count, _ = GoodMorningLog.objects.exclude(
        sent_at__date=today
    ).delete()

    # Reset trackers
    updated = ActivityTracker.objects.all().update(
        consecutive_available_count=0,
        last_status=None,
    )

    print(f"Deleted old PresenceLog records: {deleted_presence_count}")
    print(f"Deleted old GoodMorningLog records:
```

```
{deleted_good_morning_count}")
    print(f"Reset {updated} activity trackers")
```

presense_engine.py

```
import os
from dotenv import load_dotenv
from botbuilder.core import BotFrameworkAdapterSettings,
BotFrameworkAdapter

load_dotenv()

BOT_APP_ID = os.getenv("BOT_APP_ID", "").strip()
BOT_APP_PASSWORD = os.getenv("BOT_APP_PASSWORD", "").strip()
BOT_APP_TENANT_ID = os.getenv("GRAPH_TENANT_ID", "").strip()  #
same tenant for your org

# print("BOT APP ID:", BOT_APP_ID)
# print("BOT PASSWORD LOADED:", "YES" if BOT_APP_PASSWORD else
"NO")
# print("BOT PASSWORD LENGTH:", len(BOT_APP_PASSWORD) if
BOT_APP_PASSWORD else 0)
# print("BOT TENANT ID:", BOT_APP_TENANT_ID)
print("Bot Working Well")

SETTINGS = BotFrameworkAdapterSettings(
    app_id=BOT_APP_ID,
    app_password=BOT_APP_PASSWORD,
    channel_auth_tenant=BOT_APP_TENANT_ID
)
```

```
ADAPTER = BotFrameworkAdapter(SETTINGS)
teams_bot.py
from botbuilder.schema import (
    ConversationReference,
    ChannelAccount,
    ConversationAccount,
)
from botbuilder.core import TurnContext

from core.services.teams_bot import ADAPTER, BOT_APP_ID


async def send_proactive_message(employee, text: str):
    """
    Send a proactive Teams message to an employee using stored
conversation details.
    """

    if not employee.teams_user_id:
        raise Exception(f"Missing teams_user_id for
{employee.display_name}")

    if not employee.teams_conversation_id:
        raise Exception(f"Missing teams_conversation_id for
{employee.display_name}")

    if not employee.teams_service_url:
        raise Exception(f"Missing teams_service_url for
{employee.display_name}")
```

```
    conversation_reference = ConversationReference(
        channel_id="msteams",
        service_url=employee.teams_service_url,
        bot=ChannelAccount(id=BOT_APP_ID),
        user=ChannelAccount(id=employee.teams_user_id),
        conversation=ConversationAccount(
            id=employee.teams_conversation_id,
            tenant_id=employee.teams_tenant_id,
            is_group=False,
        ),
    )

    async def callback(turn_context: TurnContext):
        await turn_context.send_activity(text)

    await ADAPTER.continue_conversation(
        conversation_reference,
        callback,
        BOT_APP_ID,
    )
```

teams_sender.py

```
from django.contrib import admin
from core.models import Employee, PresenceLog, ActivityTracker,
ReminderLog, GoodMorningLog


@admin.register(Employee)
class EmployeeAdmin(admin.ModelAdmin):
```

```
    list_display = (
        "display_name",
        "email",
        "is_active",
        "reminder_enabled",
        "reminder_interval_minutes",
    )
    search_fields = ("display_name", "email")
    list_filter = ("is_active", "reminder_enabled")


@admin.register(PresenceLog)
class PresenceLogAdmin(admin.ModelAdmin):
    list_display = ("employee", "status", "checked_at")
    search_fields = ("employee__display_name",)
    list_filter = ("status", "checked_at")


@admin.register(ActivityTracker)
class ActivityTrackerAdmin(admin.ModelAdmin):
    list_display = ("employee", "consecutive_available_count",
"last_status")
    search_fields = ("employee__display_name",)


@admin.register(ReminderLog)
class ReminderLogAdmin(admin.ModelAdmin):
    list_display = ("employee", "sent_at", "message")
    search_fields = ("employee__display_name", "message")
```

```
    list_filter = ("sent_at",)

@admin.register(GoodMorningLog)
class GoodMorningLogAdmin(admin.ModelAdmin):
    list_display = ("employee", "sent_at", "message")
    search_fields = ("employee__display_name", "message")
    list_filter = ("sent_at",)
```

admin.py

```
import os
from django.apps import AppConfig


class CoreConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'core'

    def ready(self):
        # Prevent duplicate scheduler start in Django dev autoreload
        if os.environ.get("RUN_MAIN") == "true":
            from core.scheduler import start
            start()
```

apps.py

```
import uuid
from django.db import models


class Employee(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4,
editable=False)
```

```
    azure_user_id = models.CharField(max_length=255, unique=True)

    display_name = models.CharField(max_length=255)
    email = models.EmailField()

    teams_user_id = models.CharField(max_length=500, null=True,
blank=True)
    teams_conversation_id = models.TextField(null=True, blank=True)
    teams_tenant_id = models.CharField(max_length=255, null=True,
blank=True)
    teams_service_url = models.TextField(null=True, blank=True)

    is_active = models.BooleanField(default=True)
    reminder_enabled = models.BooleanField(default=True)

    reminder_interval_minutes = models.IntegerField(default=60)

    def __str__(self):
        return self.display_name


class PresenceLog(models.Model):
    employee = models.ForeignKey(Employee,
on_delete=models.CASCADE)
    status = models.CharField(max_length=50)
    checked_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
```

```
        return f"{self.employee.display_name} - {self.status} @
{self.checked_at}"


class ActivityTracker(models.Model):
    employee = models.OneToOneField(Employee,
on_delete=models.CASCADE)
    consecutive_available_count = models.IntegerField(default=0)
    last_status = models.CharField(max_length=50, null=True,
blank=True)
    reminder_sent = models.BooleanField(default=False)

    def __str__(self):
        return f"{self.employee.display_name} tracker"


class ReminderLog(models.Model):
    employee = models.ForeignKey(Employee,
on_delete=models.CASCADE)
    sent_at = models.DateTimeField(auto_now_add=True)
    message = models.TextField()

    def __str__(self):
        return f"Reminder to {self.employee.display_name} @
{self.sent_at}"

class GoodMorningLog(models.Model):
    employee = models.ForeignKey(
        "Employee",
```

```
        on_delete=models.CASCADE,
        related_name="good_morning_logs"
    )
    message = models.TextField()
    sent_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-sent_at"]
        verbose_name = "Good Morning Log"
        verbose_name_plural = "Good Morning Logs"

    def __str__(self):
        return f"{self.employee.display_name} - {self.sent_at:%Y-%m-%d
%H:%M}"
```

models.py

```
import logging
from apscheduler.schedulers.background import BackgroundScheduler

from core.services.presence_engine import (
    check_employee_presence,
    send_good_morning_messages,
    reset_daily_trackers,
)

logger = logging.getLogger(__name__)

scheduler = BackgroundScheduler(timezone="Asia/Kolkata")
```

```
def start():
    if not scheduler.running:
        # Reset trackers before workday starts
        scheduler.add_job(
            reset_daily_trackers,
            trigger='cron',
            day_of_week='mon-fri',
            hour=9,
            minute=30,
            id='reset_daily_trackers_job',
            replace_existing=True,
        )

        # Good morning at 10:00 AM Mon-Fri
        scheduler.add_job(
            send_good_morning_messages,
            trigger='cron',
            day_of_week='mon-fri',
            hour=10,
            minute=0,
            id='good_morning_job',
            replace_existing=True,
        )

        # Presence check every 2 minutes during work hours
        # 8:00 AM to 6:58 PM
        scheduler.add_job(
            check_employee_presence,
            trigger='cron',
```

```
            day_of_week='mon-fri',
            hour='8-18',
            minute='*/2',
            id='presence_check_job',
            replace_existing=True,
        )

        scheduler.start()
        logger.info("Scheduler started: reset + good morning + presence
checks")
scheduler.py
from rest_framework import serializers
from core.models import Employee, PresenceLog, ReminderLog


class EmployeeSerializer(serializers.ModelSerializer):
    class Meta:
        model = Employee
        fields = "__all__"


class PresenceLogSerializer(serializers.ModelSerializer):
    class Meta:
        model = PresenceLog
        fields = "__all__"


class ReminderLogSerializer(serializers.ModelSerializer):
    class Meta:
```

```
        model = ReminderLog
        fields = "__all__"
serializers.py
from django.urls import path
from core.views import (
    RunPresenceCheck,
    TestPresence,
    EmployeeListCreate,
    bot_messages,
)

urlpatterns = [
    path("employees/", EmployeeListCreate.as_view()),
    path("test-presence/", TestPresence.as_view()),
    path("run-check/", RunPresenceCheck.as_view()),
    path("bot/messages/", bot_messages),
]
urls.py
import json

from asgiref.sync import sync_to_async

from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status

from django.http import JsonResponse
from django.views.decorators.csrf import csrf_exempt
```

```
from botbuilder.schema import Activity
from botbuilder.core import TurnContext

from core.models import Employee
from core.serializers import EmployeeSerializer
from core.services.presence_engine import check_employee_presence
from core.services.graph import get_user_presence
from core.services.teams_bot import ADAPTER
from core.services.employee_service import
get_or_create_employee_from_teams


class EmployeeListCreate(APIView):

    def get(self, request):
        employees = Employee.objects.all().order_by("display_name")
        serializer = EmployeeSerializer(employees, many=True)
        return Response(serializer.data)

    def post(self, request):
        serializer = EmployeeSerializer(data=request.data)

        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data,
status=status.HTTP_201_CREATED)

        return Response(serializer.errors,
status=status.HTTP_400_BAD_REQUEST)
```

```
class TestPresence(APIView):

    def get(self, request):
        employee = Employee.objects.first()

        if not employee:
            return Response({"error": "No employee found"}, status=404)

        data = get_user_presence(employee.azure_user_id)
        return Response(data)


class RunPresenceCheck(APIView):

    def post(self, request):
        check_employee_presence()
        return Response({"message": "Presence check executed"})


@sync_to_async
def sync_get_or_create_employee(
    aad_object_id,
    teams_user_id,
    conversation_id,
    tenant_id,
    service_url,
    fallback_name,
```

```
):
    return get_or_create_employee_from_teams(
        aad_object_id=aad_object_id,
        teams_user_id=teams_user_id,
        conversation_id=conversation_id,
        tenant_id=tenant_id,
        service_url=service_url,
        fallback_name=fallback_name,
    )


@csrf_exempt
async def bot_messages(request):
    if request.method != "POST":
        return JsonResponse({"error": "Only POST allowed"}, status=405)

    try:
        body = json.loads(request.body)
        activity = Activity().deserialize(body)
        auth_header = request.headers.get("Authorization", "")

        async def call_bot_logic(turn_context: TurnContext):
            activity_type = turn_context.activity.type
            user_name = getattr(turn_context.activity.from_property,
"name", None)
            teams_user_id = getattr(turn_context.activity.from_property,
"id", None)
            aad_object_id = getattr(turn_context.activity.from_property,
"aad_object_id", None)
```

```
        conversation_id = getattr(turn_context.activity.conversation,
"id", None)
        tenant_id = getattr(turn_context.activity.conversation,
"tenant_id", None)
        service_url = turn_context.activity.service_url

        print("\n------ Teams Activity Received ------")
        print("Activity Type:", activity_type)
        print("User Name:", user_name)
        print("Teams User ID:", teams_user_id)
        print("AAD Object ID:", aad_object_id)
        print("Conversation ID:", conversation_id)
        print("Tenant ID:", tenant_id)
        print("Service URL:", service_url)
        print("-----------------------------------\n")

        # Auto-register / update employee if AAD object ID exists
        if aad_object_id:
            try:
                employee, created = await sync_get_or_create_employee(
                    aad_object_id=aad_object_id,
                    teams_user_id=teams_user_id,
                    conversation_id=conversation_id,
                    tenant_id=tenant_id,
                    service_url=service_url,
                    fallback_name=user_name,
                )
```

```
                if created:
                    print(f"Auto-created employee: {employee.display_name} ({employee.email})")
                else:
                    print(f"Updated Teams conversation details for {employee.display_name}")

            except Exception as emp_error:
                print(f"Employee auto-registration failed: {emp_error}")
        else:
            print("No AAD Object ID received from Teams activity")

        if activity_type in ["conversationUpdate", "installationUpdate"]:
            print(f"{activity_type} received - registration handled, no welcome message sent.")
            return

        # Handle normal user messages only
        if activity_type == "message":
            user_text = (turn_context.activity.text or "").strip()
            user_text_lower = user_text.lower()

            print("Message:", user_text)

            safe_name = user_name or "there"

            if user_text_lower in ["hi", "hello", "test"]:
                reply = (
                    f"Hello {safe_name} 👋\n"
```

```
                    f"You are connected to Wellness Reminder Bot.\n"
                    f"I will automatically send wellness reminders during
working hours."
                )
            else:
                reply = (
                    f"Hello {safe_name} 👋\n"
                    f"Your chat is connected successfully.\n"
                    f"You will receive automatic wellness reminders during
working hours."
                )

            try:
                await turn_context.send_activity(reply)
                print("Reply sent successfully")
            except Exception as send_error:
                print("BOT SEND ERROR (message):", str(send_error))
                # Do not raise -> prevents Teams retry duplication

        await ADAPTER.process_activity(
            activity,
            auth_header,
            call_bot_logic
        )

        return JsonResponse({"status": "ok"}, status=200)

    except Exception as e:
        print("BOT MESSAGE ERROR:", str(e))
```

```
        # Return 200 to avoid Teams retry storms during debugging
        return JsonResponse(
            {"status": "received_with_error", "error": str(e)},
            status=200
        )
views.py
"""
Django settings for wellnessreminder project.

Generated by 'django-admin startproject' using Django 5.2.11.

For more information on this file, see
https://docs.djangoproject.com/en/5.2/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/5.2/ref/settings/
"""

import os
from pathlib import Path
from dotenv import load_dotenv

BASE_DIR = Path(__file__).resolve().parent.parent

load_dotenv(BASE_DIR / ".env")

DEBUG = os.getenv("DEBUG", "True") == "True"

GRAPH_CLIENT_ID = os.getenv("GRAPH_CLIENT_ID")
```

```
GRAPH_CLIENT_SECRET = os.getenv("GRAPH_CLIENT_SECRET")
GRAPH_TENANT_ID = os.getenv("GRAPH_TENANT_ID")

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent


# Quick-start development settings - unsuitable for production
# See
https://docs.djangoproject.com/en/5.2/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = 'django-insecure-
f@a+&a%uw2)h(%g+z8a93q5c@6_y6q!52k8eg1gyzs$akwkx))'


ALLOWED_HOSTS = [
    "127.0.0.1",
    "localhost",
    "unsecured-elsewhere-lea.ngrok-free.dev"
]


# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
```

```
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',
    'corsheaders',
    'core.apps.CoreConfig',
]

MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware',
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'wellnessreminder.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
```

```
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'wellnessreminder.wsgi.application'


# Database
# https://docs.djangoproject.com/en/5.2/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}


# Password validation
# https://docs.djangoproject.com/en/5.2/ref/settings/#auth-password-
validators

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidat
```

```
or',
    },
    {
        'NAME':
'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME':
'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME':
'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]


# Internationalization
# https://docs.djangoproject.com/en/5.2/topics/i18n/

LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'Asia/Kolkata'

USE_I18N = True

USE_TZ = True
```

```
# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/5.2/howto/static-files/

STATIC_URL = '/static/'

# Default primary key field type
# https://docs.djangoproject.com/en/5.2/ref/settings/#default-auto-
field

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
CORS_ALLOW_ALL_ORIGINS = True

# CELERY_BROKER_URL = "redis://127.0.0.1:6379/0"
# CELERY_ACCEPT_CONTENT = ["json"]
# CELERY_TASK_SERIALIZER = "json"

# from celery.schedules import crontab

# CELERY_BEAT_SCHEDULE = {
#     "check-presence-every-10-minutes": {
#         "task": "core.tasks.run_presence_check_task",
#         "schedule": crontab(minute="*/10"),
#     },
# }
```

settings.py

```
from django.contrib import admin
from django.urls import path, include
from django.conf import settings
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('core.urls')),
]
```

main project urls
this is my project code can i deploy it or i need to make any changes for scheduler.py can i update it only programatically no docker no redis only celery or celery beat programatic solution for production

Yes — **you can deploy this project**, but **NOT in the current scheduler setup for production** if you keep starting APScheduler inside `apps.py`.

Short answer (important)

❌ Current production approach is not safe

Your current code starts APScheduler from:

- `core/apps.py` → `ready()`
- which calls `core.scheduler.start()`

This works in **local dev**, but in **production (Gunicorn / multiple workers / reloads / multiple dynos)** it can:

- start **multiple schedulers**
- send **duplicate reminders**
- send **duplicate Good Morning messages**
- create duplicate logs

This is a very common issue with APScheduler inside Django app startup. `django-apscheduler` docs also warn that multiple worker processes can cause duplicate execution, and recommend running a **single**

dedicated scheduler process instead of starting it from app import/startup. PyPI +1

Your exact question:

"No Docker, no Redis, only Celery / Celery Beat programmatic solution for production?"

Important truth:

❌ Celery cannot properly run without a broker

Celery **requires a message transport / broker** (normally Redis or RabbitMQ). The official docs say Celery requires a message transport; Redis and RabbitMQ are the fully supported brokers. Celery Documen... +1

So:

- **Celery worker + Celery Beat without Redis/RabbitMQ = not a proper production setup**
- `celery beat` only schedules tasks
- `celery worker` still needs a **broker** to receive tasks

Therefore:

If you want:

- ❌ No Docker
- ❌ No Redis
- ❌ No RabbitMQ
- ❌ Only programmatic

Then the **best production option for your project is NOT Celery**.

Best production solution for your current project (recommended)

✅ Use APScheduler, but run it as a separate single process

This is the best option for your use case.

Architecture:

Run **2 processes** in production:

1. **Django web app**
   - `gunicorn wellnessreminder.wsgi:application ...`
2. **Dedicated scheduler process**
   - `python manage.py run_scheduler`

This gives you:

- only **one scheduler instance**
- no duplicate jobs
- no Redis
- no Docker
- no Celery
- works well for your current reminder bot use case

What you MUST change in your code before production

---

1) ❌ Remove scheduler auto-start from `apps.py`

Your current `core/apps.py`

```
import os
from django.apps import AppConfig


class CoreConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'core'

    def ready(self):
        # Prevent duplicate scheduler start in Django dev autoreload
        if os.environ.get("RUN_MAIN") == "true":
            from core.scheduler import start
            start()
```

Problem:

This only helps in `runserver` dev mode.

In production with Gunicorn:

- each worker may import Django app
- `ready()` may run multiple times
- scheduler may start multiple times

✅ Replace `core/apps.py` with this:

```python
from django.apps import AppConfig


class CoreConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
```

Python Run

---

2) ✅ Keep `core/scheduler.py`, but improve it slightly

Your current `scheduler.py` is mostly fine, but I recommend:

- use `max_instances=1`
- use `coalesce=True`
- use `misfire_grace_time`
- correct your reset time comment mismatch

✅ Recommended `core/scheduler.py`

```python
import logging
from apscheduler.schedulers.blocking import BlockingScheduler

from core.services.presence_engine import (
    check_employee_presence,
    send_good_morning_messages,
    reset_daily_trackers,
)
```

```python
logger = logging.getLogger(__name__)

scheduler = BlockingScheduler(timezone="Asia/Kolkata")


def start():
    # Reset trackers before workday starts
    scheduler.add_job(
        reset_daily_trackers,
        trigger="cron",
        day_of_week="mon-fri",
        hour=7,
        minute=55,
        id="reset_daily_trackers_job",
        replace_existing=True,
        max_instances=1,
        coalesce=True,
        misfire_grace_time=300,
    )

    # Good morning at 10:00 AM Mon-Fri
    scheduler.add_job(
        send_good_morning_messages,
        trigger="cron",
        day_of_week="mon-fri",
        hour=10,
        minute=0,
        id="good_morning_job",
        replace_existing=True,
        max_instances=1,
        coalesce=True,
        misfire_grace_time=300,
    )

    # Presence check every 2 minutes during work hours
```

```
    # 8:00 AM to 6:58 PM Mon-Fri
    scheduler.add_job(
        check_employee_presence,
        trigger="cron",
        day_of_week="mon-fri",
        hour="8-18",
        minute="*/2",
        id="presence_check_job",
        replace_existing=True,
        max_instances=1,
        coalesce=True,
        misfire_grace_time=120,
    )

    logger.info("Scheduler started: reset + good morning + presence checks")
    scheduler.start()
```

---

3) ✅ Create a Django management command for scheduler

This is the **correct production pattern**.

Create folders:

```
core/
  management/
    __init__.py
    commands/
      __init__.py
      run_scheduler.py
```

File: `core/management/commands/run_scheduler.py`

```
from django.core.management.base import BaseCommand
from core.scheduler import start


class Command(BaseCommand):
    help = "Run APScheduler for wellness reminders"

    def handle(self, *args, **options):
        self.stdout.write(self.style.SUCCESS("Starting wellness scheduler..."))
        start()
```

---

4) ✅ Fix a wrong import mismatch in your pasted code (very important)

Your file names and imports are inconsistent in the message you pasted.

In `core/scheduler.py` you have:

```
from core.services.presence_engine import (
    check_employee_presence,
    send_good_morning_messages,
    reset_daily_trackers,
)
```

But in your screenshot / pasted list:

You wrote file as:

- `presense_engine.py` ❌ (spelling wrong)

It should be:

- `presence_engine.py` ✅

If your actual filename is really `presense_engine.py`, then production will fail with:

- `ModuleNotFoundError: No module named core.services.presence_engine`

Fix:

Rename file:

```
core/services/presense_engine.py
```

to

```
core/services/presence_engine.py
```

---

5) ⚠️ Another file naming/import mismatch (very important)

In `presence_engine.py` you import:

```python
```

But from your pasted content:

- `teams_sender.py` contains **admin code** ❌
- `teams_bot.py` contains `send_proactive_message()` ❌

So your files are swapped/misnamed in the message.

It should be like this:

`core/services/teams_sender.py` ✅

```
from botbuilder.schema import (
    ConversationReference,
    ChannelAccount,
    ConversationAccount,
)
from botbuilder.core import TurnContext

from core.services.teams_bot import ADAPTER, BOT_APP_ID


async def send_proactive_message(employee, text: str):
    """
    Send a proactive Teams message to an employee using stored conversation details.
    """

    if not employee.teams_user_id:
        raise Exception(f"Missing teams_user_id for {employee.display_name}")

    if not employee.teams_conversation_id:
        raise Exception(f"Missing teams_conversation_id for {employee.display_name}")
```

```
    if not employee.teams_service_url:
        raise Exception(f"Missing teams_service_url for {employee.display_name}")

    conversation_reference = ConversationReference(
        channel_id="msteams",
        service_url=employee.teams_service_url,
        bot=ChannelAccount(id=BOT_APP_ID),
        user=ChannelAccount(id=employee.teams_user_id),
        conversation=ConversationAccount(
            id=employee.teams_conversation_id,
            tenant_id=employee.teams_tenant_id,
            is_group=False,
        ),
    )

    async def callback(turn_context: TurnContext):
        await turn_context.send_activity(text)

    await ADAPTER.continue_conversation(
        conversation_reference,
        callback,
        BOT_APP_ID,
    )
```

`core/admin.py` must contain admin code only

Right now your pasted content shows admin code inside `teams_sender.py`.

Move that code back to `core/admin.py`.

---

6) ✅ Keep your `teams_bot.py` as adapter-only

Your current `teams_bot.py` (based on pasted content) should only contain:

```
import os
from dotenv import load_dotenv
from botbuilder.core import BotFrameworkAdapterSettings, BotFrameworkAdapter

load_dotenv()

BOT_APP_ID = os.getenv("BOT_APP_ID", "").strip()
BOT_APP_PASSWORD = os.getenv("BOT_APP_PASSWORD", "").strip()
BOT_APP_TENANT_ID = os.getenv("GRAPH_TENANT_ID", "").strip()

print("Bot Working Well")

SETTINGS = BotFrameworkAdapterSettings(
    app_id=BOT_APP_ID,
    app_password=BOT_APP_PASSWORD,
    channel_auth_tenant=BOT_APP_TENANT_ID
)

ADAPTER = BotFrameworkAdapter(SETTINGS)
```

That is fine.

---

7) ⚠️ Your `reset_daily_trackers()` comment and schedule do not match

In `presence_engine.py`:

Python Run

But in `scheduler.py` you currently scheduled:

```python
hour=9,
```

Python Run

That means:

- comments say **7:55 AM**
- actual job runs **9:30 AM**

Recommendation:

Use **7:55 AM** (before workday starts), because your working hours begin at 8 AM.

So set:

```python
hour=7,
```

Python Run

---

8) ⚠️ Your `check_employee_presence` schedule is okay, but note this behavior

Current schedule:

```python
hour='8-18',
```

Python Run

This means:

- 8:00 to 18:58
- every 2 minutes
- good, because 19:00 excluded

And your logic also checks:

```python
```

Python Run

That matches. Good ✅

---

9) ⚠️ SQLite is okay for small demo, but for production prefer PostgreSQL

Right now:

```python
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
```

```
        'NAME': BASE_DIR / 'db.sqlite3',
    }
```

Python Run

For real production:

Use **PostgreSQL** instead of SQLite because:

- concurrent reads/writes from:
  - web app
  - scheduler process
- better reliability
- better locking behavior
- safer for growth

SQLite can still work for:

- testing
- internal demo
- single small deployment

But for production bot reminders:

👉 **PostgreSQL strongly recommended**

---

10) ⚠️ Add logging instead of `print()`

For production, replace many `print()` calls with `logger.info() / logger.warning() / logger.exception()`.

This is not mandatory to deploy, but highly recommended.

Example:

```python
import logging
logger = logging.getLogger(__name__)

logger.info("Reminder sent to %s", employee.display_name)
logger.exception("Failed to send Teams reminder")
```

---

11) ⚠️ Your `views.py` bot endpoint is okay, but safe for production only if URL is public HTTPS

Your endpoint:

```python
```

For Teams production:

- must be publicly reachable
- HTTPS required
- ngrok only for testing
- production domain should be stable

Your `ALLOWED_HOSTS` currently includes:

```python
```

Python Run

That is okay for testing, not final production.

---

12) Can you do “programmatic only” scheduling without Docker/Redis?

✅ YES — best answer for your case:

Use **APScheduler + dedicated process**.

Commands in production:

Web app:

Bash

Scheduler (separate terminal / service):

Bash

---

13) If you REALLY want Celery without Redis?

Honest answer:

❌ Not recommended for production

Celery officially expects a broker. Redis and RabbitMQ are the feature-complete transports. Celery Documen... +1

There are old/experimental alternatives, but for your case:

- fragile
- not worth it
- harder than APScheduler
- worse than your current needs

So:

Do NOT move to Celery right now
unless you are ready to use:

- Redis **or**
- RabbitMQ

---

14) Final recommended production structure for YOU

Best production stack for your current project:

- **Django + Gunicorn**
- **APScheduler in separate management command**
- **PostgreSQL** (recommended)
- **No Docker required**
- **No Redis required**
- **No Celery required**

---

15) Exact files you should have after cleanup

`core/apps.py`

```
from django.apps import AppConfig


class CoreConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'core'
```

`core/management/commands/run_scheduler.py`

```
from django.core.management.base import BaseCommand
from core.scheduler import start


class Command(BaseCommand):
    help = "Run APScheduler for wellness reminders"

    def handle(self, *args, **options):
        self.stdout.write(self.style.SUCCESS("Starting wellness scheduler..."))
        start()
```

`core/scheduler.py`

(Use the improved version I gave above)

---

16) My final verdict on your project

✅ Can you deploy now?

YES, but only after these required fixes:

Required before production:

- ☐ Remove scheduler startup from `apps.py`
- ☐ Run scheduler via **management command** only
- ☐ Ensure only **one scheduler process**
- ☐ Fix filename typo: `presense_engine.py` → `presence_engine.py`
- ☐ Fix file mix-up: `teams_sender.py` vs `admin.py`
- ☐ Correct reset time to 7:55 AM (or update comment)
- ☐ Prefer PostgreSQL instead of SQLite
- ☐ Use stable HTTPS URL (not ngrok) for Teams bot
- ☐ Add logging

---

17) Best answer to your Celery question in one line

✅ Recommended:

Use APScheduler as a separate dedicated process (management command), not Celery.

❌ Not recommended:

Celery/Celery Beat without Redis/RabbitMQ

---

18) What I recommend you do RIGHT NOW

If you want, I can give you the **exact final production-ready code files** (copy-paste ready) for:

1. `core/apps.py`
2. `core/scheduler.py`
3. `core/management/commands/run_scheduler.py`
4. corrected `core/services/presence_engine.py`
5. corrected `core/services/teams_sender.py`
6. corrected `core/admin.py`
7. optional `systemd` service files for Linux server (no Docker)

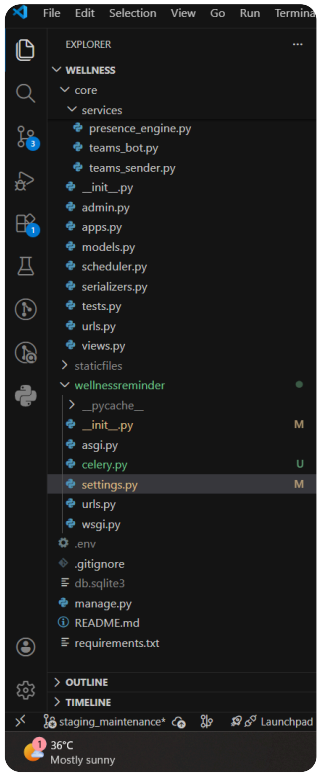
If you want, I'll give you STEP 1 → STEP 10 exact copy-paste production setup next, with:

- exact folder creation
- exact code
- exact commands
- exact deployment commands

Say:

"give me final production-ready files step by step"

Sources



instead of any above steps i have made some research and installed redis lically on my system
in one cmd i have redis-cli open in one cmd i have redis-server open in vs code terminal i activated worker "PS D:\NOTIFY\Wellness> & d:\NOTIFY\environment\Scripts\Activate.ps1
(environment) PS D:\NOTIFY\Wellness> celery -A wellnessreminder worker --pool=solo -l info
Bot Working Well

 -------------- celery@DESKTOP-1J9LPI2 v5.6.2 (recovery)
--- ***** -----

```
-- ******* ---- Windows-10-10.0.26200-SP0 2026-03-24 18:16:55
- *** --- * ---
- ** ---------- [config]
- ** ---------- .> app:         wellnessreminder:0x19435cc3710
- ** ---------- .> transport:   redis://127.0.0.1:6379//
- ** ---------- .> results:     redis://127.0.0.1:6379/
- *** --- * --- .> concurrency: 8 (solo)
-- ******* ---- .> task events: OFF (enable -E to monitor tasks in this
worker)
--- ***** -----
 -------------- [queues]
                .> celery           exchange=celery(direct) key=celery


[tasks]


[2026-03-24 18:16:55,502: INFO/MainProcess] Connected to
redis://127.0.0.1:6379//
[2026-03-24 18:16:55,502: INFO/MainProcess] mingle: searching for
neighbors
[2026-03-24 18:16:56,526: INFO/MainProcess] mingle: all alone
[2026-03-24 18:16:56,565: INFO/MainProcess] celery@DESKTOP-
1J9LPI2 ready."
```

in my main project cmd i have runserver for django

```
"System check identified no issues (0 silenced).
March 24, 2026 - 18:17:05
Django version 5.2.11, using settings 'wellnessreminder.settings'
Starting development server at http://127.0.0.1:8000/
```

Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/5.2/howto/deployment/
Sangram status: Available
Working detected -> Sangram: 9"
code file i added/updated below
import os

from celery import Celery

# Set the default Django settings module for the 'celery' program.
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'wellnessreminder.settings')

app = Celery('wellnessreminder')

# Using a string here means the worker doesn't have to serialize
# the configuration object to child processes.
# - namespace='CELERY' means all celery-related configuration keys
# should have a `CELERY_` prefix.
app.config_from_object('django.conf:settings', namespace='CELERY')

# Load task modules from all registered Django apps.
app.autodiscover_tasks()
celery.py
from .celery import app as celery_app

__all__ = ('celery_app',)
__init__.py

CELERY_BROKER_URL = "redis://127.0.0.1:6379"
CELERY_RESULT_BACKEND = "redis://127.0.0.1:6379"
those lines in settings.py
now i want you to tell me how i can use it instead of apscheduler like how to do it proparlly how will it work i am new to celery and redis so please exlain and tell me proparlly

Perfect — **now you're on the right path** ✅
Since you already installed **Redis locally** and started a **Celery worker**, we can now move your project from **APScheduler → Celery + Celery Beat** properly.

And I can already see one important thing from your logs:

> `Sangram status: Available`
> `Working detected -> Sangram: 9`

That means **your old APScheduler is STILL running from** `apps.py`, not Celery yet.
Because your Celery worker currently shows:

```
[tasks]
```

...with **no tasks listed**, which means Celery is running but **it has nothing to execute yet**.

🔥 Most important summary first

Right now:

- ✅ Redis is running
- ✅ Celery worker is running
- ✅ Django is running
- ❌ You have **not created any Celery tasks yet**
- ❌ You have **not started Celery Beat yet**
- ❌ APScheduler is probably **still running from** `apps.py`

So the next goal is:

✅ Final production/dev architecture with Celery

You should run **4 things** (for local dev):

1. **Redis server**
2. **Django app**
3. **Celery worker**
4. **Celery Beat** (scheduler)

How Celery works (simple explanation)

Think of it like this:

1) Celery Beat = Scheduler

This is the replacement for APScheduler.

It says:

- run `reset_daily_trackers` at 7:55 AM
- run `send_good_morning_messages` at 10:00 AM
- run `check_employee_presence` every 2 minutes

But Beat **does NOT run the task itself**.

---

2) Celery Worker = Task Executor

The worker listens to Redis queue.

When Beat says:

> "Run `check_employee_presence` now"

Beat puts a message in Redis.

Then the **worker picks it up** and executes the actual function.

---

3) Redis = Queue / Broker

Redis is the middleman.

- Beat sends task → Redis
- Worker reads task ← Redis

⚠️ VERY IMPORTANT: Why your current setup is not actually using Celery yet

Your worker log shows:

```
[tasks]
```

No tasks listed.

That means:

- Celery started
- but it didn't discover any `@shared_task` or `@app.task`

And your Django log is still printing:

```
Sangram status: Available
Working detected -> Sangram: 9
```

That means **APScheduler in `apps.py` is still executing**.

✅ What we will do now (step by step)

We will do this properly:

1. **Stop APScheduler auto-start**
2. **Create `core/tasks.py`**
3. **Move scheduled jobs into Celery tasks**
4. **Configure `CELERY_BEAT_SCHEDULE` in `settings.py`**
5. **Run Celery Beat**
6. **Test manually**
7. **Production command setup**

✅ STEP 1: STOP APScheduler completely

Your current `core/apps.py` is likely still this:

Python Run

```python
import os
from django.apps import AppConfig


class CoreConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'core'

    def ready(self):
        # Prevent duplicate scheduler start in Django dev autoreload
        if os.environ.get("RUN_MAIN") == "true":
            from core.scheduler import start
            start()
```

❌ This must be removed now.

Replace `core/apps.py` with this:

Python Run

```python
from django.apps import AppConfig
```

```
class CoreConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'core'
```

Why?

Because if you keep this:

- APScheduler runs inside Django
- Celery also runs
- you get **duplicate reminders**
- and debugging becomes confusing

✅ STEP 2: Keep your existing business logic in `presence_engine.py`

This file is already good.

Your `core/services/presence_engine.py` contains:

- `check_employee_presence()`
- `send_good_morning_messages()`
- `reset_daily_trackers()`

That's perfect.

👉 **Do NOT move this logic.**
We will simply call these functions from Celery tasks.

✅ STEP 3: Create `core/tasks.py`

This is the file Celery will auto-discover.

Create file:

```
core/tasks.py
```

Put this exact code:

Python Run

```python
from celery import shared_task

from core.services.presence_engine import (
    check_employee_presence,
    send_good_morning_messages,
    reset_daily_trackers,
)


@shared_task(name="core.tasks.run_presence_check_task")
def run_presence_check_task():
    """
    Celery task to check employee presence and send wellness reminders.
    """
    check_employee_presence()


@shared_task(name="core.tasks.send_good_morning_task")
def send_good_morning_task():
    """
    Celery task to send good morning messages.
```

```
    """
    send_good_morning_messages()


@shared_task(name="core.tasks.reset_daily_trackers_task")
def reset_daily_trackers_task():
    """
    Celery task to reset daily trackers and clean old logs.
    """
    reset_daily_trackers()
```

✅ Why this file is important

Celery automatically looks for:

- `tasks.py` in installed Django apps

Because you already have:

Python Run

```python
app.autodiscover_tasks()
```

inside `wellnessreminder/celery.py`

So once `core/tasks.py` exists, your worker should show:

```
[tasks]
  . core.tasks.run_presence_check_task
  . core.tasks.send_good_morning_task
  . core.tasks.reset_daily_trackers_task
```

✅ STEP 4: Update `wellnessreminder/settings.py` properly

You currently added:

```python
CELERY_BROKER_URL = "redis://127.0.0.1:6379"
CELERY_RESULT_BACKEND = "redis://127.0.0.1:6379"
```

That's good start.
Now add **all proper Celery settings**.

Add this to `settings.py` (replace your current Celery section)

```python
from celery.schedules import crontab

CELERY_BROKER_URL = "redis://127.0.0.1:6379/0"
CELERY_RESULT_BACKEND = "redis://127.0.0.1:6379/0"

CELERY_ACCEPT_CONTENT = ["json"]
CELERY_TASK_SERIALIZER = "json"
CELERY_RESULT_SERIALIZER = "json"

CELERY_TIMEZONE = "Asia/Kolkata"
CELERY_ENABLE_UTC = False

CELERY_BEAT_SCHEDULE = {
    # Reset trackers before workday starts
```

```
    "reset-daily-trackers": {
        "task": "core.tasks.reset_daily_trackers_task",
        "schedule": crontab(minute=55, hour=7, day_of_week="1-5"),  # Mon-Fri 7:55 AM
    },

    # Good morning at 10:00 AM Mon-Fri
    "send-good-morning": {
        "task": "core.tasks.send_good_morning_task",
        "schedule": crontab(minute=0, hour=10, day_of_week="1-5"),  # Mon-Fri 10:00 AM
    },

    # Presence check every 2 minutes during work hours
    "check-presence-every-2-minutes": {
        "task": "core.tasks.run_presence_check_task",
        "schedule": crontab(minute="*/2", hour="8-18", day_of_week="1-5"),  # Mon-Fri 8:00-18:58
    },
}
```

✅ Why `day_of_week="1-5"`?

In Celery crontab:

- 1 = Monday
- 5 = Friday

So:

- `1-5` = Mon-Fri

✅ STEP 5: You no longer need APScheduler

Since Celery Beat will now schedule everything:

You can keep `core/scheduler.py` in project for now, but:

- **do not use it**
- **do not import it**
- **do not start it anywhere**

Best practice:

Either:

- keep it unused temporarily, or
- delete it later after Celery is fully working

✅ STEP 6: Restart everything in the correct order

Because you changed code, stop all terminals first.

Stop:

- Django server
- Celery worker
- Redis server (optional)
- redis-cli (optional)

✅ STEP 7: Start services properly

Terminal 1 — Redis

```bash
redis-server
```

Terminal 2 — Django

Activate env, then:

```bash
python manage.py runserver
```

Terminal 3 — Celery Worker (Windows-friendly)

Since you're on Windows, use **solo pool**:

```bash
celery -A wellnessreminder worker --pool=solo -l info
```

Expected output should now include tasks like:

```
[tasks]
  . core.tasks.reset_daily_trackers_task
```

```
  . core.tasks.run_presence_check_task
  . core.tasks.send_good_morning_task
```

If you still see empty `[tasks]`, something is wrong.

Terminal 4 — Celery Beat

This is the scheduler (VERY important)

```bash
celery -A wellnessreminder beat -l info
```

This is what actually replaces APScheduler.

Without this command:

- worker runs
- but **nothing gets scheduled automatically**

✅ STEP 8: Very important — how to know it's working

Beat terminal should show logs like:

```
Scheduler: Sending due task check-presence-every-2-minutes (core.tasks.run_presence_check_ta
```

Worker terminal should show logs like:

```
Task core.tasks.run_presence_check_task[...] received
Task core.tasks.run_presence_check_task[...] succeeded
```

That means:

- Beat scheduled task
- Redis queued task
- Worker executed task

✅ STEP 9: Manual testing before waiting for schedule

You should not wait until 7:55 or 10:00 for testing.

Best quick test:

Run task manually from Django shell.

Command:

```bash
python manage.py shell
```

Then:

```python
from core.tasks import run_presence_check_task
run_presence_check_task.delay()
```

What happens:

- `.delay()` sends task to Redis
- Worker picks it up
- Your presence logic runs

Test other tasks:

Python Run

```python
from core.tasks import send_good_morning_task
send_good_morning_task.delay()
```

Python Run

```python
from core.tasks import reset_daily_trackers_task
reset_daily_trackers_task.delay()
```

✅ STEP 10: Your current worker output explained

You showed:

```
[tasks]
```

with nothing below it.

That means one of these is true:

- `core/tasks.py` does not exist yet ❌
- syntax/import error inside `core/tasks.py` ❌
- `core` app not in `INSTALLED_APPS` (but yours is already there) ✅
- worker started before `tasks.py` was created, and not restarted ❌

✅ STEP 11: Why "Bot Working Well" appears in Celery worker

Your worker log shows:

```
Bot Working Well
```

That comes from:

```python
print("Bot Working Well")
```

inside `core/services/teams_bot.py`

This happens because:

- Celery imports your Django app
- tasks import `presence_engine`
- `presence_engine` imports `send_proactive_message`
- `send_proactive_message` imports `teams_bot`
- `teams_bot` executes module-level code

So that print is expected.

But for production:

👉 Replace that `print("Bot Working Well")` with nothing (or logger), because every import prints it.

✅ STEP 12: Best project structure now

You already have:

- `wellnessreminder/celery.py` ✅
- `wellnessreminder/__init__.py` ✅

Now add:

- `core/tasks.py` ✅

That is the minimum correct Celery structure.

✅ STEP 13: Recommended final code changes summary

---

1) `core/apps.py`

Python Run

```python
from django.apps import AppConfig


class CoreConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'core'
```

---

2) `core/tasks.py`

Python Run

```python
from celery import shared_task

from core.services.presence_engine import (
    check_employee_presence,
    send_good_morning_messages,
    reset_daily_trackers,
)


@shared_task(name="core.tasks.run_presence_check_task")
```

ChatGPT Share ...

```python


@shared_task(name="core.tasks.send_good_morning_task")
def send_good_morning_task():
    send_good_morning_messages()


@shared_task(name="core.tasks.reset_daily_trackers_task")
```

```
def reset_daily_trackers_task():
    reset_daily_trackers()
```

---

3) `wellnessreminder/settings.py`

Python Run

```python
from celery.schedules import crontab

CELERY_BROKER_URL = "redis://127.0.0.1:6379/0"
CELERY_RESULT_BACKEND = "redis://127.0.0.1:6379/0"

CELERY_ACCEPT_CONTENT = ["json"]
CELERY_TASK_SERIALIZER = "json"
CELERY_RESULT_SERIALIZER = "json"

CELERY_TIMEZONE = "Asia/Kolkata"
CELERY_ENABLE_UTC = False

CELERY_BEAT_SCHEDULE = {
    "reset-daily-trackers": {
        "task": "core.tasks.reset_daily_trackers_task",
        "schedule": crontab(minute=55, hour=7, day_of_week="1-5"),
    },
    "send-good-morning": {
        "task": "core.tasks.send_good_morning_task",
        "schedule": crontab(minute=0, hour=10, day_of_week="1-5"),
    },
    "check-presence-every-2-minutes": {
        "task": "core.tasks.run_presence_check_task",
        "schedule": crontab(minute="*/2", hour="8-18", day_of_week="1-5"),
```

```
    },
}
```

✅ STEP 14: Important production advice (VERY important)

In production, do NOT run:

```bash
celery -A wellnessreminder worker -B
```

Because `-B` means:

- worker + beat together in one process
- okay for dev
- **not recommended for production**

Celery docs explicitly note `-B` is convenient for development, but in production beat should run separately, and only **one beat instance** should exist to avoid duplicate schedules. docs.celeryq.dev +1

Correct production:

- 1 Django web process
- 1 Celery worker process
- 1 Celery Beat process
- 1 Redis process/service

✅ STEP 15: How Celery Beat stores schedule state

By default, Celery Beat uses a **persistent local schedule file** (usually `celerybeat-schedule`) to track last-run times. Celery's default scheduler is `PersistentScheduler`, and the schedule filename can be configured.

docs.celeryq.dev +1

So in your project folder you may see a file like:

```
celerybeat-schedule
celerybeat-schedule.db
```

That's normal.

✅ STEP 16: Single Beat instance rule (very important)

Celery docs say:

> Only **one scheduler (beat)** should run for a schedule, otherwise you can get duplicate executions.
>
> docs.celeryq.dev +1

So never run:

- two `celery beat` terminals
- or two servers both running beat

Otherwise:

- duplicate reminders
- duplicate Good Morning messages

✅ STEP 17: Windows note (important for you)

You are on Windows, so:

Worker:

Use:

```bash
celery -A wellnessreminder worker --pool=solo -l info
```

This is correct for Windows/local dev.

Celery's `solo` pool is single-threaded / single-process, which is why it's commonly used on Windows/dev when prefork isn't available. docs.celeryq.dev +1

Small note:

Your log says:

```
concurrency: 8 (solo)
```

That looks odd because `solo` effectively executes one at a time. Don't worry too much; with `solo`, task execution is still single-process.

✅ STEP 18: My recommendation for your project specifically

For **your wellness bot**, this is the best setup:

- Keep business logic in `presence_engine.py`
- Create `core/tasks.py`
- Schedule with `CELERY_BEAT_SCHEDULE`

- Remove APScheduler startup from `apps.py`
- Run:
  - Redis
  - Django
  - Celery worker
  - Celery beat

This is a **good production direction**.

🚨 VERY IMPORTANT: Why your Django terminal currently shows live presence logs

Because APScheduler is still active.

You wrote:

```
Sangram status: Available
Working detected -> Sangram: 9
```

That should **NOT** appear just because Django started.

If it appears immediately on `runserver`, that means:

- `core/apps.py` still auto-starts scheduler
- or `scheduler.py` is imported somewhere and started

Fix this first before doing anything else.

✅ Recommended next immediate action (do this now)

1. Update `core/apps.py`

Remove APScheduler startup completely.

2. Create `core/tasks.py`

Add the 3 Celery tasks.

3. Update `settings.py`

Add full `CELERY_BEAT_SCHEDULE`.

4. Restart all terminals.

5. Run:

- `redis-server`
- `python manage.py runserver`
- `celery -A wellnessreminder worker --pool=solo -l info`
- `celery -A wellnessreminder beat -l info`

6. Test with:

Python Run

```python
from core.tasks import run_presence_check_task
run_presence_check_task.delay()
```

✅ Final verdict

Can you replace APScheduler with Celery now?

YES — 100%

Since you already installed Redis, **Celery + Celery Beat is now the correct way**.

🎯 Best next step from me

I can now give you the **exact final copy-paste ready code** for:

1. `core/apps.py`
2. `core/tasks.py`
3. `wellnessreminder/settings.py`
4. optional cleanup of `core/scheduler.py`
5. exact commands to run
6. how to verify in logs
7. how to deploy in production on Windows/Linux

If you want, reply exactly:

“give me final celery files copy paste ready”

And I’ll give you the **final corrected code file by file** (no confusion, direct copy-paste).

Sources

now see in one terminal i see ths "Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

D:\NOTIFY\Wellness>"D:\NOTIFY\environment\Scripts\activate.bat"

(environment) D:\NOTIFY\Wellness>celery -A wellnessreminder beat -l info
celery beat v5.6.2 (recovery) is starting.
Bot Working Well
__ - ... __ - _
LocalTime -> 2026-03-25 11:36:21
Configuration ->
. broker -> redis://127.0.0.1:6379/0
. loader -> celery.loaders.app.AppLoader
. scheduler -> celery.beat.PersistentScheduler
. db -> celerybeat-schedule
. logfile -> [stderr]@%INFO
. maxinterval -> 5.00 minutes (300s)
[2026-03-25 11:36:21,866: INFO/MainProcess] beat: Starting...
[2026-03-25 11:38:00,015: INFO/MainProcess] Scheduler: Sending due task check-presence-every-2-minutes (core.tasks.run_presence_check_task)
[2026-03-25 11:40:00,000: INFO/MainProcess] Scheduler: Sending due task check-presence-every-2-minutes (core.tasks.run_presence_check_task)
[2026-03-25 11:42:00,006: INFO/MainProcess] Scheduler: Sending due

```
task check-presence-every-2-minutes
(core.tasks.run_presence_check_task)
[2026-03-25 11:44:00,003: INFO/MainProcess] Scheduler: Sending due
task check-presence-every-2-minutes
(core.tasks.run_presence_check_task)
[2026-03-25 11:46:00,000: INFO/MainProcess] Scheduler: Sending due
task check-presence-every-2-minutes
(core.tasks.run_presence_check_task)
[2026-03-25 11:48:00,010: INFO/MainProcess] Scheduler: Sending due
task check-presence-every-2-minutes
(core.tasks.run_presence_check_task)
[2026-03-25 11:50:00,002: INFO/MainProcess] Scheduler: Sending due
task check-presence-every-2-minutes
(core.tasks.run_presence_check_task)
[2026-03-25 11:52:00,012: INFO/MainProcess] Scheduler: Sending due
task check-presence-every-2-minutes
(core.tasks.run_presence_check_task)
[2026-03-25 11:54:00,002: INFO/MainProcess] Scheduler: Sending due
task check-presence-every-2-minutes
(core.tasks.run_presence_check_task)
[2026-03-25 11:56:00,001: INFO/MainProcess] Scheduler: Sending due
task check-presence-every-2-minutes
(core.tasks.run_presence_check_task)
[2026-03-25 11:58:00,008: INFO/MainProcess] Scheduler: Sending due
task check-presence-every-2-minutes
(core.tasks.run_presence_check_task)
[2026-03-25 12:00:00,007: INFO/MainProcess] Scheduler: Sending due
task check-presence-every-2-minutes
(core.tasks.run_presence_check_task)
```

[2026-03-25 12:02:00,000: INFO/MainProcess] Scheduler: Sending due task check-presence-every-2-minutes (core.tasks.run_presence_check_task)
[2026-03-25 12:04:00,004: INFO/MainProcess] Scheduler: Sending due task check-presence-every-2-minutes (core.tasks.run_presence_check_task)
[2026-03-25 12:06:00,000: INFO/MainProcess] Scheduler: Sending due task check-presence-every-2-minutes (core.tasks.run_presence_check_task)
", in main runserver i see this "(environment) D:\NOTIFY\Wellness>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

Bot Working Well
System check identified no issues (0 silenced).
March 25, 2026 - 11:52:38
Django version 5.2.11, using settings 'wellnessreminder.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/5.2/howto/deployment/
[25/Mar/2026 11:58:13] "GET /admin/core/presencelog/ HTTP/1.1" 200 57971
[25/Mar/2026 11:58:13] "GET /admin/jsi18n/ HTTP/1.1" 200 3342
" in another terminal i see this "(environment) PS D:\NOTIFY\Wellness>

```
celery -A wellnessreminder worker --pool=solo -l info
Bot Working Well

 -------------- celery@DESKTOP-1J9LPI2 v5.6.2 (recovery)
--- ***** -----
-- ******* ---- Windows-10-10.0.26200-SP0 2026-03-25 12:05:42
- *** --- * ---
- ** ---------- [config]
- ** ---------- .> app:         wellnessreminder:0x23b8b0e1b90
- ** ---------- .> transport:   redis://127.0.0.1:6379/0
- ** ---------- .> results:     redis://127.0.0.1:6379/0
- *** --- * --- .> concurrency: 8 (solo)
-- ******* ---- .> task events: OFF (enable -E to monitor tasks in this
worker)
--- ***** -----
 -------------- [queues]
                .> celery           exchange=celery(direct) key=celery


[tasks]
  . core.tasks.reset_daily_trackers_task
  . core.tasks.run_presence_check_task
  . core.tasks.send_good_morning_task

[2026-03-25 12:05:42,391: INFO/MainProcess] Connected to
redis://127.0.0.1:6379/0
[2026-03-25 12:05:42,391: INFO/MainProcess] mingle: searching for
neighbors
[2026-03-25 12:05:43,414: INFO/MainProcess] mingle: all alone
```

```
[2026-03-25 12:05:43,441: INFO/MainProcess] celery@DESKTOP-1J9LPI2 ready.
[2026-03-25 12:06:00,005: INFO/MainProcess] Task core.tasks.run_presence_check_task[9bd1c11c-2017-48e4-a575-aeaab46eb052] received
[2026-03-25 12:06:01,333: WARNING/MainProcess] Sangram status: Available
[2026-03-25 12:06:01,499: WARNING/MainProcess] Working detected -> Sangram: 19
[2026-03-25 12:06:02,576: WARNING/MainProcess] Senthil status: Away
[2026-03-25 12:06:02,697: WARNING/MainProcess] Resetting counter for Senthil
[2026-03-25 12:06:03,888: WARNING/MainProcess] sejal status: Available
[2026-03-25 12:06:04,023: WARNING/MainProcess] Working detected -> sejal: 19
[2026-03-25 12:06:05,070: WARNING/MainProcess] Harshad status: Available
[2026-03-25 12:06:05,191: WARNING/MainProcess] Working detected -> Harshad: 3
[2026-03-25 12:06:06,016: WARNING/MainProcess] Nitin More status: Busy
[2026-03-25 12:06:06,123: WARNING/MainProcess] Working detected -> Nitin More: 2
[2026-03-25 12:06:07,074: WARNING/MainProcess] Vaibhav Deshpande status: Offline
[2026-03-25 12:06:07,202: WARNING/MainProcess] Resetting counter for Vaibhav Deshpande
[2026-03-25 12:06:08,478: WARNING/MainProcess] Akriti Bharti status:
```

Away
[2026-03-25 12:06:08,611: WARNING/MainProcess] Resetting counter for Akriti Bharti
[2026-03-25 12:06:09,704: WARNING/MainProcess] River Dvijam status: Busy
[2026-03-25 12:06:09,827: WARNING/MainProcess] Working detected -> River Dvijam: 20
[2026-03-25 12:06:11,041: WARNING/MainProcess] Irsha Sovani status: Busy
[2026-03-25 12:06:11,220: WARNING/MainProcess] Working detected -> Irsha Sovani: 1
[2026-03-25 12:06:12,136: WARNING/MainProcess] Ritika Juneja status: Available
[2026-03-25 12:06:11,220: WARNING/MainProcess] Working detected -> Irsha Sovani: 1
[2026-03-25 12:06:12,136: WARNING/MainProcess] Ritika Juneja status: Available
[2026-03-25 12:06:12,136: WARNING/MainProcess] Ritika Juneja status: Available
[2026-03-25 12:06:12,242: WARNING/MainProcess] Working detected -> Ritika Juneja: 2
[2026-03-25 12:06:13,653: WARNING/MainProcess] Arrfan Dannge status: Available
[2026-03-25 12:06:13,653: WARNING/MainProcess] Arrfan Dannge status: Available
[2026-03-25 12:06:13,794: WARNING/MainProcess] Working detected -> Arrfan Dannge: 8
[2026-03-25 12:06:15,036: WARNING/MainProcess] Janhavi Bendre status: Away

[2026-03-25 12:06:15,166: WARNING/MainProcess] Resetting counter for Janhavi Bendre
[2026-03-25 12:06:16,191: WARNING/MainProcess] Vikram Ahir status: Available
[2026-03-25 12:06:16,362: WARNING/MainProcess] Working detected -> Vikram Ahir: 13
[2026-03-25 12:06:17,255: WARNING/MainProcess] Dnyaneshwar Kolhe status: Available
[2026-03-25 12:06:17,375: WARNING/MainProcess] Working detected -> Dnyaneshwar Kolhe: 11
[2026-03-25 12:06:18,545: WARNING/MainProcess] Renuka Chavan status: Available
[2026-03-25 12:06:18,660: WARNING/MainProcess] Working detected -> Renuka Chavan: 27
[2026-03-25 12:06:18,749: INFO/MainProcess] Task core.tasks.run_presence_check_task[9bd1c11c-2017-48e4-a575-aeaab46eb052] succeeded in 18.75s: None
", this is cmd redis-server "Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\SangramBhosale>redis-server
[1280] 25 Mar 11:34:58.525 # oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
[1280] 25 Mar 11:34:58.525 # Redis version=5.0.14.1, bits=64, commit=ec77f72d, modified=0, pid=1280, just started
[1280] 25 Mar 11:34:58.525 # Warning: no config file specified, using the default config. In order to specify a config file use redis-server /path/to/redis.conf

```
[1280] 25 Mar 11:34:58.529 # Could not create server TCP listening
socket *:6379: bind: An operation was attempted on something that is
not a socket.
"
now my code files are below
"""
Django settings for wellnessreminder project.

Generated by 'django-admin startproject' using Django 5.2.11.

For more information on this file, see
https://docs.djangoproject.com/en/5.2/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/5.2/ref/settings/
"""

import os
from pathlib import Path
from dotenv import load_dotenv
from celery.schedules import crontab

BASE_DIR = Path(__file__).resolve().parent.parent

load_dotenv(BASE_DIR / ".env")

DEBUG = os.getenv("DEBUG", "True") == "True"

GRAPH_CLIENT_ID = os.getenv("GRAPH_CLIENT_ID")
```

```
GRAPH_CLIENT_SECRET = os.getenv("GRAPH_CLIENT_SECRET")
GRAPH_TENANT_ID = os.getenv("GRAPH_TENANT_ID")

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent


# Quick-start development settings - unsuitable for production
# See
https://docs.djangoproject.com/en/5.2/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = 'django-insecure-
f@a+&a%uw2)h(%g+z8a93q5c@6_y6q!52k8eg1gyzs$akwkx))'


ALLOWED_HOSTS = [
    "127.0.0.1",
    "localhost",
    "unsecured-elsewhere-lea.ngrok-free.dev"
]


# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
```

```
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',
    'corsheaders',
    'core.apps.CoreConfig',
]

MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware',
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'wellnessreminder.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
```

```
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'wellnessreminder.wsgi.application'


# Database
# https://docs.djangoproject.com/en/5.2/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}


# Password validation
# https://docs.djangoproject.com/en/5.2/ref/settings/#auth-password-
validators

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidat
```

```
or',
    },
    {
        'NAME':
'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME':
'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME':
'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]


# Internationalization
# https://docs.djangoproject.com/en/5.2/topics/i18n/

LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'Asia/Kolkata'

USE_I18N = True

USE_TZ = True
```

```
# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/5.2/howto/static-files/

STATIC_URL = '/static/'

# Default primary key field type
# https://docs.djangoproject.com/en/5.2/ref/settings/#default-auto-field

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
CORS_ALLOW_ALL_ORIGINS = True


CELERY_BROKER_URL = "redis://127.0.0.1:6379/0"
CELERY_RESULT_BACKEND = "redis://127.0.0.1:6379/0"

CELERY_ACCEPT_CONTENT = ["json"]
CELERY_TASK_SERIALIZER = "json"
CELERY_RESULT_SERIALIZER = "json"

CELERY_TIMEZONE = "Asia/Kolkata"
CELERY_ENABLE_UTC = False

CELERY_BEAT_SCHEDULE = {
    # Reset trackers before workday starts
    "reset-daily-trackers": {
        "task": "core.tasks.reset_daily_trackers_task",
        "schedule": crontab(minute=55, hour=7, day_of_week="1-5"),  # Mon-Fri 7:55 AM
```

```
    },

    # Good morning at 10:00 AM Mon-Fri
    "send-good-morning": {
        "task": "core.tasks.send_good_morning_task",
        "schedule": crontab(minute=0, hour=10, day_of_week="1-5"),  #
Mon-Fri 10:00 AM
    },

    # Presence check every 2 minutes during work hours
    "check-presence-every-2-minutes": {
        "task": "core.tasks.run_presence_check_task",
        "schedule": crontab(minute="*/2", hour="8-18", day_of_week="1-
5"),  # Mon-Fri 8:00-18:58
    },
}
```

settings.py

```
import os

from celery import Celery

# Set the default Django settings module for the 'celery' program.
os.environ.setdefault('DJANGO_SETTINGS_MODULE',
'wellnessreminder.settings')

app = Celery('wellnessreminder')

# Using a string here means the worker doesn't have to serialize
# the configuration object to child processes.
```

```
# - namespace='CELERY' means all celery-related configuration keys
#   should have a CELERY_ prefix.
app.config_from_object('django.conf:settings', namespace='CELERY')

# Load task modules from all registered Django apps.
app.autodiscover_tasks()
celery.py
from .celery import app as celery_app

__all__ = ('celery_app',)
__init__.py
from celery import shared_task

from core.services.presence_engine import (
    check_employee_presence,
    send_good_morning_messages,
    reset_daily_trackers,
)


@shared_task(name="core.tasks.run_presence_check_task")
def run_presence_check_task():
    """
    Celery task to check employee presence and send wellness
reminders.
    """
    check_employee_presence()
```

```
@shared_task(name="core.tasks.send_good_morning_task")
def send_good_morning_task():
    """
    Celery task to send good morning messages.
    """
    send_good_morning_messages()


@shared_task(name="core.tasks.reset_daily_trackers_task")
def reset_daily_trackers_task():
    """
    Celery task to reset daily trackers and clean old logs.
    """
    reset_daily_trackers()
```

core/tasks.py

```
import logging
from apscheduler.schedulers.background import BackgroundScheduler

from core.services.presence_engine import (
    check_employee_presence,
    send_good_morning_messages,
    reset_daily_trackers,
)

logger = logging.getLogger(__name__)

scheduler = BackgroundScheduler(timezone="Asia/Kolkata")
```

```
def start():
    if not scheduler.running:
        # Reset trackers before workday starts
        scheduler.add_job(
            reset_daily_trackers,
            trigger='cron',
            day_of_week='mon-fri',
            hour=9,
            minute=40,
            id='reset_daily_trackers_job',
            replace_existing=True,
        )

        # Good morning at 10:00 AM Mon-Fri
        scheduler.add_job(
            send_good_morning_messages,
            trigger='cron',
            day_of_week='mon-fri',
            hour=10,
            minute=0,
            id='good_morning_job',
            replace_existing=True,
        )

        # Presence check every 2 minutes during work hours
        # 8:00 AM to 6:58 PM
        scheduler.add_job(
            check_employee_presence,
            trigger='cron',
```

```
            day_of_week='mon-fri',
            hour='8-18',
            minute='*/2',
            id='presence_check_job',
            replace_existing=True,
        )

        scheduler.start()
        logger.info("Scheduler started: reset + good morning + presence
checks")
scheduler.py which i dont use now
# import os
# from django.apps import AppConfig


# class CoreConfig(AppConfig):
#     default_auto_field = 'django.db.models.BigAutoField'
#     name = 'core'

#     def ready(self):
#         # Prevent duplicate scheduler start in Django dev autoreload
#         if os.environ.get("RUN_MAIN") == "true":
#             from core.scheduler import start
#             start()

from django.apps import AppConfig


class CoreConfig(AppConfig):
```

```
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'core'
core/apps.py
import math

from asgiref.sync import async_to_sync
from django.utils import timezone

from core.models import Employee, PresenceLog, ActivityTracker,
ReminderLog, GoodMorningLog
from core.services.graph import get_user_presence
from core.services.teams_sender import send_proactive_message


WORKING_STATUSES = [
    "Available",
    "Busy",
    "InACall",
    "InAMeeting",
]

GOOD_MORNING_ALLOWED_STATUSES = [
    "Available",
    "Busy",
    "InACall",
    "InAMeeting",
]

CHECK_INTERVAL_MINUTES = 2
```

```
WELLNESS_REMINDER_MESSAGES = [
    "✨ Small break, big benefit — take 2 minutes to move and recharge.",
    "💆 Stretch reminder! Roll your neck and shoulders gently to reduce stiffness.",
    "🖥️ Screen break time — rest your eyes and blink a few times slowly.",
    "🧍 Quick posture check! Sit tall or stand up and stretch for a moment.",
    "🌬️ Take a deep breath — a short breathing break can improve focus.",
    "💧 Hydration check! Take a sip of water and relax your shoulders.",
    "🧘 Short break time! Stand up, stretch, and refresh your body for 2 minutes.",
    "👀 Give your eyes a break — look away from the screen for a few moments.",
    "🚶 Time to move a little! A short walk can help reset your focus.",
    "🌿 Pause for a moment — breathe deeply and reset your energy.",
    "💺 Sitting too long? Stand up and loosen your back and legs for a minute.",
    "🚰 Wellness check! Drink water and take a mindful pause before continuing.",
    "⏰ Time for a quick wellness break! Please drink some water and stretch a little.",
    "🚰 Friendly reminder: hydrate, stretch, and return refreshed.",
]

GOOD_MORNING_MESSAGES = [
```

```
    "😊 Good morning! Wishing you a smooth workday with good focus, good health, and good energy.",
    "🌞 Good morning! Hope you start your day with energy, focus, and a healthy routine.",
    "🌸 Good morning! Wishing you a calm mind, good posture, and a productive day ahead.",
    "💧 Good morning! Stay hydrated, take mindful breaks, and have a wonderful workday.",
    "🌿 Good morning! A fresh start for a fresh day — work well and remember to take care of yourself.",
    "✨ Good morning! Small healthy habits today can make a big difference — have a great day ahead.",
    "🌞 Good morning! Wishing you a productive and healthy workday. I'll gently remind you to take wellness breaks during the day.",
    "🧘 Good morning! Begin your day with focus and balance. I'll be here to remind you to take short wellness breaks.",
    "🚶 Good morning! Wishing you a positive day filled with productivity and healthy movement.",
    "🌸 Good morning! Stay active, stretch often, and make today a great one.",
    "🌈 Good morning! Hope your day is productive, peaceful, and full of positive energy.",
    "💼 Good morning! Let's make today productive and healthy — don't forget to stretch and hydrate.",
]


def is_within_working_hours():
    """
```

```
    Monday-Friday, 8:00 AM to 7:00 PM (Asia/Kolkata)
    """
    now = timezone.localtime()
    weekday = now.weekday()  # Monday=0, Sunday=6
    current_hour = now.hour

    is_weekday = weekday < 5
    within_hours = 8 <= current_hour < 19

    return is_weekday and within_hours


def get_next_reminder_message(employee):
    """
    Return a reminder message that has not yet been used today for this
employee.
    If all messages have been used today, fallback to first message.
    """
    today = timezone.localdate()

    used_messages_today = set(
        ReminderLog.objects.filter(
            employee=employee,
            sent_at__date=today
        ).values_list("message", flat=True)
    )

    for message in WELLNESS_REMINDER_MESSAGES:
        if message not in used_messages_today:
```

```
            return message

    # Fallback if all messages used in same day (unlikely but safe)
    return WELLNESS_REMINDER_MESSAGES[0]

def get_next_good_morning_message(employee):
    """
    Return a Good Morning message for this employee.

    Rules:
    - Only one Good Morning per employee per day
    - Since old GoodMorningLogs are deleted daily, do NOT rely on
history
    - Rotate message using today's date + employee id
    """
    today = timezone.localdate()

    # Prevent duplicate Good Morning on same day
    already_sent_today = GoodMorningLog.objects.filter(
        employee=employee,
        sent_at__date=today
    ).exists()

    if already_sent_today:
        return None

    # Rotate messages daily using date + employee id
    # Works for UUID primary keys also
    day_number = today.toordinal()
```

```
    if employee.id:
        employee_seed = employee.id.int if hasattr(employee.id, "int") else
int(employee.id)
    else:
        employee_seed = 0

    index = (day_number + employee_seed) %
len(GOOD_MORNING_MESSAGES)
    return GOOD_MORNING_MESSAGES[index]

def has_teams_chat_initialized(employee):
    """
    Check whether employee has opened the bot at least once and
conversation details exist.
    """
    return (
        bool(employee.teams_user_id)
        and bool(employee.teams_conversation_id)
        and bool(employee.teams_service_url)
    )


def fetch_employee_presence(employee, save_log=True):
    """
    Fetch employee presence safely.
    Returns status string or None if fetch fails.

    If save_log=True, also stores PresenceLog.
```

```
    """
    try:
        presence_data = get_user_presence(employee.azure_user_id)
        status = presence_data.get("availability")

        print(f"{employee.display_name} status: {status}")

        if save_log:
            PresenceLog.objects.create(
                employee=employee,
                status=status or "Unknown"
            )

        return status

    except Exception as e:
        print(f"Presence fetch failed for {employee.display_name}: {e}")
        return None


def check_employee_presence():
    """
    Runs every 2 minutes during working hours.
    Sends reminder after continuous working duration reaches employee
threshold.
    """
    if not is_within_working_hours():
        print("Outside working hours. Skipping presence check.")
        return
```

```
    employees = Employee.objects.filter(
        is_active=True,
        reminder_enabled=True,
    )

    for employee in employees:
        # User must have initialized chat at least once
        if not has_teams_chat_initialized(employee):
            print(f"Skipping {employee.display_name}: no Teams
conversation details yet")
            continue

        status = fetch_employee_presence(employee, save_log=True)

        if status is None:
            continue

        tracker, _ =
ActivityTracker.objects.get_or_create(employee=employee)

        if status in WORKING_STATUSES:
            tracker.consecutive_available_count += 1
            print(f"Working detected -> {employee.display_name}:
{tracker.consecutive_available_count}")
        else:
            print(f"Resetting counter for {employee.display_name}")
            tracker.consecutive_available_count = 0
```

```
        # Example:
        # 60 mins / 2 mins = 30 checks
        # Use ceil so reminders are never sent earlier than configured
interval
        required_checks = max(
            1,
            math.ceil(employee.reminder_interval_minutes /
CHECK_INTERVAL_MINUTES)
        )

        if tracker.consecutive_available_count >= required_checks:
            reminder_message = get_next_reminder_message(employee)

            print(f"Reminder triggered for {employee.display_name}")
            print(f"Reminder message: {reminder_message}")

            try:
                async_to_sync(send_proactive_message)(employee,
reminder_message)
                print(f"Reminder sent to Teams for
{employee.display_name}")

                # Create log only after successful send
                ReminderLog.objects.create(
                    employee=employee,
                    message=reminder_message
                )
                print(f"Reminder log created for {employee.display_name}")
```

```
            # Reset only after successful send
            tracker.consecutive_available_count = 0

        except Exception as e:
            print(f"Failed to send Teams reminder to
{employee.display_name}: {e}")

    tracker.last_status = status
    tracker.save()


def send_good_morning_messages():
    """
    Sends Good Morning message only to active employees who:
    - have initialized Teams chat
    - are currently online/working (not offline/away)
    - have NOT already received Good Morning today

    Runs at scheduled time Mon-Fri.
    """
    now = timezone.localtime()
    weekday = now.weekday()

    if weekday >= 5:
        print("Weekend. Skipping good morning messages.")
        return

    employees = Employee.objects.filter(
        is_active=True,
```

```
        reminder_enabled=True,
    )

    for employee in employees:
        if not has_teams_chat_initialized(employee):
            print(f"Skipping good morning for {employee.display_name}: no
Teams chat initialized")
            continue

        # Prevent duplicate Good Morning in same day
        good_morning_message =
get_next_good_morning_message(employee)
        if good_morning_message is None:
            print(f"Skipping good morning for {employee.display_name}:
already sent today")
            continue

        # Fetch live presence before sending Good Morning
        status = fetch_employee_presence(employee, save_log=True)

        if status is None:
            print(f"Skipping good morning for {employee.display_name}:
presence unavailable")
            continue

        if status not in GOOD_MORNING_ALLOWED_STATUSES:
            print(
                f"Skipping good morning for {employee.display_name}: "
                f"status is {status} (not eligible)"
```

```
            )
            continue

        try:
            async_to_sync(send_proactive_message)(employee,
good_morning_message)
            print(f"Good morning message sent to
{employee.display_name}")
            print(f"Good morning message: {good_morning_message}")

            # Create log only after successful send
            GoodMorningLog.objects.create(
                employee=employee,
                message=good_morning_message
            )
            print(f"Good morning log created for
{employee.display_name}")

        except Exception as e:
            print(f"Failed to send good morning to
{employee.display_name}: {e}")


def reset_daily_trackers():
    """
    Reset all activity counters before workday starts.
    Also delete all old PresenceLogs and GoodMorningLogs
    so only today's logs remain.
    Runs at 7:55 AM Mon-Fri.
```

```
    """
    today = timezone.localdate()

    # Delete old presence logs
    deleted_presence_count, _ = PresenceLog.objects.exclude(
        checked_at__date=today
    ).delete()

    # Delete old good morning logs
    deleted_good_morning_count, _ = GoodMorningLog.objects.exclude(
        sent_at__date=today
    ).delete()

    # Reset trackers
    updated = ActivityTracker.objects.all().update(
        consecutive_available_count=0,
        last_status=None,
    )

    print(f"Deleted old PresenceLog records: {deleted_presence_count}")
    print(f"Deleted old GoodMorningLog records:
{deleted_good_morning_count}")
    print(f"Reset {updated} activity trackers")
```

presense_engine.py

```
import os
from dotenv import load_dotenv
from botbuilder.core import BotFrameworkAdapterSettings,
BotFrameworkAdapter
```

```
load_dotenv()

BOT_APP_ID = os.getenv("BOT_APP_ID", "").strip()
BOT_APP_PASSWORD = os.getenv("BOT_APP_PASSWORD", "").strip()
BOT_APP_TENANT_ID = os.getenv("GRAPH_TENANT_ID", "").strip()  #
same tenant for your org

# print("BOT APP ID:", BOT_APP_ID)
# print("BOT PASSWORD LOADED:", "YES" if BOT_APP_PASSWORD else
"NO")
# print("BOT PASSWORD LENGTH:", len(BOT_APP_PASSWORD) if
BOT_APP_PASSWORD else 0)
# print("BOT TENANT ID:", BOT_APP_TENANT_ID)
print("Bot Working Well")

SETTINGS = BotFrameworkAdapterSettings(
    app_id=BOT_APP_ID,
    app_password=BOT_APP_PASSWORD,
    channel_auth_tenant=BOT_APP_TENANT_ID
)

ADAPTER = BotFrameworkAdapter(SETTINGS)
teams_bot.py
from botbuilder.schema import (
    ConversationReference,
    ChannelAccount,
    ConversationAccount,
)
from botbuilder.core import TurnContext
```

```
from core.services.teams_bot import ADAPTER, BOT_APP_ID


async def send_proactive_message(employee, text: str):
    """
    Send a proactive Teams message to an employee using stored
conversation details.
    """

    if not employee.teams_user_id:
        raise Exception(f"Missing teams_user_id for
{employee.display_name}")

    if not employee.teams_conversation_id:
        raise Exception(f"Missing teams_conversation_id for
{employee.display_name}")

    if not employee.teams_service_url:
        raise Exception(f"Missing teams_service_url for
{employee.display_name}")

    conversation_reference = ConversationReference(
        channel_id="msteams",
        service_url=employee.teams_service_url,
        bot=ChannelAccount(id=BOT_APP_ID),
        user=ChannelAccount(id=employee.teams_user_id),
        conversation=ConversationAccount(
            id=employee.teams_conversation_id,
```

```
            tenant_id=employee.teams_tenant_id,
            is_group=False,
        ),
    )

    async def callback(turn_context: TurnContext):
        await turn_context.send_activity(text)

    await ADAPTER.continue_conversation(
        conversation_reference,
        callback,
        BOT_APP_ID,
    )
```

teams_sender.py

```
import requests
import msal
from django.conf import settings

AUTHORITY =
f"https://login.microsoftonline.com/{settings.GRAPH_TENANT_ID}"
SCOPE = ["https://graph.microsoft.com/.default"]

_app = msal.ConfidentialClientApplication(
    settings.GRAPH_CLIENT_ID,
    authority=AUTHORITY,
    client_credential=settings.GRAPH_CLIENT_SECRET,
)
```

```
def get_access_token():
    """
    Acquire app-only access token using MSAL.
    MSAL handles token caching automatically.
    """
    result = _app.acquire_token_for_client(scopes=SCOPE)

    if "access_token" in result:
        return result["access_token"]

    raise Exception(f"Token error: {result}")


def get_user_presence(user_id):
    """
    Get Teams presence for a user using Azure AD Object ID.
    """
    token = get_access_token()

    url = f"https://graph.microsoft.com/v1.0/users/{user_id}/presence"

    headers = {
        "Authorization": f"Bearer {token}",
        "Content-Type": "application/json",
    }

    response = requests.get(url, headers=headers)

    if response.status_code == 200:
```

```
        return response.json()

    raise Exception(f"Graph Presence Error {response.status_code}:
{response.text}")


def get_user_profile(user_id):
    """
    Fetch user profile from Microsoft Graph using Azure AD Object ID.
    Useful for auto-registering employee on first bot message.
    """
    token = get_access_token()

    url = (
        f"https://graph.microsoft.com/v1.0/users/{user_id}"
        f"?$select=id,displayName,mail,userPrincipalName"
    )

    headers = {
        "Authorization": f"Bearer {token}",
        "Content-Type": "application/json",
    }

    response = requests.get(url, headers=headers)

    if response.status_code == 200:
        return response.json()

    raise Exception(f"Graph Profile Error {response.status_code}:
```

```
{response.text}")
graph.py
from core.models import Employee
from core.services.graph import get_user_profile


def get_or_create_employee_from_teams(
    aad_object_id,
    teams_user_id,
    conversation_id,
    tenant_id,
    service_url,
    fallback_name=None,
):
    """
    Auto-create employee when user first messages bot.
    If employee exists, update Teams conversation details.
    """
    if not aad_object_id:
        raise Exception("Missing aad_object_id from Teams activity")

    employee =
Employee.objects.filter(azure_user_id=aad_object_id).first()

    if employee:
        # Update Teams conversation details every time
        employee.teams_user_id = teams_user_id
        employee.teams_conversation_id = conversation_id
        employee.teams_tenant_id = tenant_id
```

```
        employee.teams_service_url = service_url
        employee.save(update_fields=[
            "teams_user_id",
            "teams_conversation_id",
            "teams_tenant_id",
            "teams_service_url",
        ])
        return employee, False

    # Not found -> fetch profile from Graph
    profile = get_user_profile(aad_object_id)

    display_name = profile.get("displayName") or fallback_name or
"Unknown User"
    email = (
        profile.get("mail")
        or profile.get("userPrincipalName")
        or f"{aad_object_id}@example.com"
    )

    employee = Employee.objects.create(
        azure_user_id=aad_object_id,
        display_name=display_name,
        email=email,
        teams_user_id=teams_user_id,
        teams_conversation_id=conversation_id,
        teams_tenant_id=tenant_id,
        teams_service_url=service_url,
        is_active=True,
```

```
        reminder_enabled=True,
        reminder_interval_minutes=60,
    )

    return employee, True
```

employee_service.py
this is my code i tried to change time to reset daily trackers but it wont work and are my terminals good overall project structure is good? is this ready for deployment ? how to start it in my local like terminal flow and path tell me proparlly how to start it in my local with celery redis and my wroject workflow

Excellent work, Sangram — **your Celery + Beat + Redis flow is now basically working** ✅ 🔥

From your logs, I can confirm:

- ✅ **Celery Beat is working**
  (`Scheduler: Sending due task check-presence-every-2-minutes`)
- ✅ **Celery Worker is working**
  (`Task core.tasks.run_presence_check_task received`)
- ✅ **Worker is executing your logic correctly**
  (`Sangram status: Available`, `Working detected -> Sangram: 19`)
- ✅ **Django is NOT auto-running APScheduler anymore**
  (good — `runserver` terminal is clean now)
- ⚠️ **Redis server command shows bind error because Redis is already running elsewhere**
- ⚠️ `reset_daily_trackers` **schedule time change may not appear because of beat persistent schedule file / restart timing confusion**

- ⚠️ **Project is good for local dev/demo, but NOT fully production-ready yet**

✅ FIRST: Answer to your main questions

1) Are your terminals good overall?

YES — current terminal setup is correct

You should have these 4 things running locally:

Terminal 1 — Redis

A Redis instance running in background
(you already have one running, even though the second `redis-server` command failed)

Terminal 2 — Django

```bash
python manage.py runserver
```

Terminal 3 — Celery Worker

```bash
celery -A wellnessreminder worker --pool=solo -l info
```

Terminal 4 — Celery Beat

```bash
celery -A wellnessreminder beat -l info
```

👉 This is the **correct local development flow** for your project now.

✅ 2) Is your overall project structure good?

YES — now it is good enough structurally

Your structure is now correct:

- `wellnessreminder/celery.py` ✅
- `wellnessreminder/__init__.py` ✅
- `core/tasks.py` ✅
- `core/apps.py` no scheduler auto-start ✅
- `core/services/presence_engine.py` contains business logic ✅
- `core/services/teams_sender.py` proactive sender ✅
- `core/services/teams_bot.py` adapter setup ✅
- `core/services/graph.py` Graph API logic ✅

So overall architecture is now **much better than before**.

⚠️ 3) Why `reset_daily_trackers` time change “won’t work”?

You changed:

```python
"reset-daily-trackers": {
    "task": "core.tasks.reset_daily_trackers_task",
    "schedule": crontab(minute=55, hour=7, day_of_week="1-5"),
}
```

but you said it doesn't seem to reflect.

Most likely reason:

Celery Beat uses a persistent schedule file

Your beat log shows:

```
.scheduler -> celery.beat.PersistentScheduler
.db -> celerybeat-schedule
```

That means Celery Beat stores schedule metadata in a local file.

✅ BEST FIX (important)

Whenever you change `CELERY_BEAT_SCHEDULE`, do this:

1. Stop Celery Beat

Press `Ctrl + C`

2. Delete the beat schedule file from project folder

In your project root (`D:\NOTIFY\Wellness`), delete files if they exist:

- `celerybeat-schedule`
- `celerybeat-schedule.db`
- `celerybeat-schedule.dat`
- `celerybeat-schedule.dir`
- `celerybeat-schedule.bak`

Sometimes only one exists, sometimes multiple depending on backend/platform.

3. Start Beat again

```bash
celery -A wellnessreminder beat -l info
```

Why this helps

Celery Beat caches last-run info and sometimes schedule metadata.

So after schedule changes, safest practice is:

✅ **delete beat schedule file + restart beat**

⚠️ 4) Very important: Your Redis server error explained

You showed:

```
Could not create server TCP listening socket *:6379: bind: An operation was attempted on sor
```

What this means:

Most likely:

- Redis is **already running on port 6379**
- So starting a second Redis instance fails

This is actually consistent with your worker/beat logs, because:

- Worker connected to `redis://127.0.0.1:6379/0` ✅
- Beat is scheduling tasks via Redis ✅

So **Redis is already running somewhere**.

✅ How to verify Redis is actually running

Open a new terminal:

```bash
redis-cli ping
```

Expected:

```
PONG
```

If you get `PONG`, Redis is healthy.

✅ Best practice for local

If Redis is already running as a Windows service or background process:

👉 Do NOT start `redis-server` manually again

Just use:

```bash
redis-cli ping
```

If it returns `PONG`, you're good.

⚠️ 5) Why `Bot Working Well` prints in all terminals

You see:

- in Django
- in worker
- in beat

Because inside `core/services/teams_bot.py` you have:

```python
print("Bot Working Well")
```

That file is imported by multiple processes:

- Django process
- Celery worker process

- Celery beat process

So every process prints it.

✅ Recommendation

Remove this line completely:

```python
print("Bot Working Well")
```

This is harmless, but noisy.

✅ 6) Your current local workflow is working correctly

From your worker log:

```
Task core.tasks.run_presence_check_task[...] received
...
Sangram status: Available
Working detected -> Sangram: 19
...
Task core.tasks.run_presence_check_task[...] succeeded in 18.75s
```

This means:

Actual flow right now:

1. **Celery Beat** hits every 2 minutes

2. Beat sends task to **Redis**
3. **Celery Worker** picks task from Redis
4. Worker runs `core.tasks.run_presence_check_task`
5. That calls `check_employee_presence()`
6. It loops through employees
7. Fetches presence via Microsoft Graph
8. Updates tracker
9. Sends reminders when threshold is reached

👉 This is exactly what you wanted.

⚠️ 7) One important performance observation from your logs

Your task took:

```
succeeded in 18.75s
```

That's okay **for now**, but keep this in mind:

- You run every **2 minutes**
- Current task duration is **~19 seconds**
- That's still safe (because 19 sec < 120 sec)

So currently okay ✅

But if employees increase a lot:

- 30+ sec
- 60+ sec
- 100+ sec

Then you may need:

- batching
- parallelization
- lower API overhead
- rate-limit handling

For now: **totally fine**

⚠️ 8) Is this ready for deployment?

Short answer:

Almost — but NOT fully production-ready yet

Your Celery setup is now correct.
But your **Django settings are still not production-safe**.

🚨 9) What must be changed before production

A) `DEBUG=True` ❌

Current:

```python
DEBUG = os.getenv("DEBUG", "True") == "True"
```

If `.env` doesn't override it, production may run in debug.

Better:

In `.env` for production:

```env
DEBUG=False
```

B) Hardcoded `SECRET_KEY` ❌

Current:

```python
SECRET_KEY = 'django-insecure-...'
```

Production fix:

```python
SECRET_KEY = os.getenv("DJANGO_SECRET_KEY")
```

And in `.env`:

```env
DJANGO_SECRET_KEY=your-very-long-secret-key
```

C) SQLite in production ⚠️

Current:

```python
'django.db.backends.sqlite3'
```

For local/demo: okay

For real production: use PostgreSQL

Because you now have:

- Django web app
- Celery worker
- Celery beat
- all touching DB

SQLite can work for small demos, but for production:
👉 **PostgreSQL is strongly recommended**

D) `ALLOWED_HOSTS` needs real production domain

Current:

```python
ALLOWED_HOSTS = [
    "127.0.0.1",
    "localhost",
```

```
    "unsecured-elsewhere-lea.ngrok-free.dev"
]
```

For production add:

- your real server domain
- your real HTTPS public bot endpoint domain

E) `CORS_ALLOW_ALL_ORIGINS = True` ⚠️

Okay for dev, not ideal for production.

F) Do not use `runserver` in production ❌

Use:

- **Gunicorn** on Linux
- or **Waitress** on Windows (if you insist on Windows)
- but Linux + Gunicorn is much better

✅ 10) My honest production recommendation for YOU

Best production stack:

- **Linux server / VM**
- **Nginx**
- **Gunicorn**
- **Redis**

- Celery worker
- Celery beat
- PostgreSQL
- HTTPS public domain for Teams bot

⚠️ 11) One small but important file note

In your message you pasted:

```
presense_engine.py
```

But your screenshot earlier showed:

```
presence_engine.py
```

And your imports use:

```python
from core.services.presence_engine import ...
```

Make sure actual filename is:

```
core/services/presence_engine.py
```

NOT:

```
core/services/presense_engine.py
```

If filename is wrong, deployment/import will break.

✅ 12) Your local startup flow (exact proper order)

This is the exact correct local workflow now.

📁 Project path

```
D:\NOTIFY\Wellness
```

Terminal 1 — Activate env + verify Redis

```bat
D:\NOTIFY\environment\Scripts\activate.bat
cd /d D:\NOTIFY\Wellness
redis-cli ping
```

Expected:

```
PONG
```

If you get `PONG`

👉 Redis is already running, do NOT start `redis-server` again.

If Redis is NOT running

Then start it:

```bat
redis-server
```

Terminal 2 — Django server

```bat
D:\NOTIFY\environment\Scripts\activate.bat
cd /d D:\NOTIFY\Wellness
python manage.py runserver
```

Terminal 3 — Celery worker

```bat
D:\NOTIFY\environment\Scripts\activate.bat
cd /d D:\NOTIFY\Wellness
celery -A wellnessreminder worker --pool=solo -l info
```

Terminal 4 — Celery beat

```bat
D:\NOTIFY\environment\Scripts\activate.bat
cd /d D:\NOTIFY\Wellness
celery -A wellnessreminder beat -l info
```

✅ 13) How to restart correctly after schedule changes

Whenever you change:

- `CELERY_BEAT_SCHEDULE`
- task names
- Celery config
- timezone

Do this:

Step A — Stop worker + beat

Press `Ctrl + C` in:

- Celery worker terminal
- Celery beat terminal

Step B — Delete beat schedule file

In `D:\NOTIFY\Wellness`, delete:

- `celerybeat-schedule*`

Step C — Restart worker

```bat
celery -A wellnessreminder worker --pool=solo -l info
```

Step D — Restart beat

```bat
celery -A wellnessreminder beat -l info
```

✅ 14) How to test tasks manually (VERY useful)

Instead of waiting for exact schedule times:

Open Django shell

```Bash
python manage.py shell
```

Test presence check:

```python
from core.tasks import run_presence_check_task
run_presence_check_task.delay()
```

Test good morning:

```python
from core.tasks import send_good_morning_task
send_good_morning_task.delay()
```

Test reset:

```python
from core.tasks import reset_daily_trackers_task
reset_daily_trackers_task.delay()
```

Then watch **worker terminal**.

✅ 15) Recommended cleanup (important)

`core/scheduler.py`

You are not using APScheduler anymore.

You can keep it for now, but safest:

Either:

- delete it, OR
- leave it unused

That's fine.

`core/apps.py`

This is correct now:

```python
from django.apps import AppConfig


class CoreConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'core'
```

✅ Keep this.

✅ 16) One improvement I strongly recommend now

Your `print()` logs appear in Celery as `WARNING/MainProcess`, which is messy.

Example:

```
[WARNING/MainProcess] Sangram status: Available
```

That happens because Celery captures stdout/stderr.

Better approach:

Replace `print()` in `presence_engine.py` with Python logging.

At top:

```python
import logging
logger = logging.getLogger(__name__)
```

Then replace:

```python
print(f"{employee.display_name} status: {status}")
```

with:

```python
logger.info("%s status: %s", employee.display_name, status)
```

This is not required immediately, but strongly recommended before production.

✅ 17) Is your current system behaving correctly right now?

YES — based on your logs:

Your system is behaving correctly.

Beat:

```
Scheduler: Sending due task check-presence-every-2-minutes
```

✅ Good

Worker:

```
Task core.tasks.run_presence_check_task received
...
Task ... succeeded
```

✅ Good

Django:

No scheduler spam, only admin/web requests
✅ Good

Redis:

Broker connection successful
✅ Good

🚨 18) Final production readiness verdict

Current status:

Local development / internal testing: YES, ready

Production: NOT YET, needs hardening

Before production, do these:

- ☐ Remove `print("Bot Working Well")`
- ☐ Use `.env` for `SECRET_KEY`
- ☐ Set `DEBUG=False`
- ☐ Move from SQLite → PostgreSQL
- ☐ Restrict `ALLOWED_HOSTS`
- ☐ Restrict CORS
- ☐ Use Gunicorn/Waitress instead of `runserver`
- ☐ Run Celery worker + beat as background services
- ☐ Use a stable public HTTPS bot URL (not ngrok)

✅ 19) My direct answer to “is it ready?”

For local testing / demo:

YES — your setup is good now

For real production:

NO — needs 1 final cleanup round

But your **Celery + Redis architecture is now correct**. That was the hardest part, and you've done it. 👏

🎯 What I recommend next (best next step)

I can now give you the **FINAL production-ready version** of these exact files, copy-paste ready:

1. `settings.py` (production-safe version)
2. `core/tasks.py`
3. `core/apps.py`
4. `core/services/presence_engine.py` (**with logging instead of print**)
5. `core/services/teams_bot.py` (remove noisy print)
6. exact **Windows local start commands**
7. exact **Linux production commands**
8. exact **deploy checklist**

Reply exactly:

"give me final cleaned production-ready files"

and I'll give you **the exact final code** file-by-file, with:

- what to keep
- what to remove
- what command to run
- how to deploy properly without confusion

to skip ngrok step i need to host it somewhere

Yes — if you want to skip ngrok, you need to host it somewhere publicly reachable over HTTPS.

For **Microsoft Teams bot**, your bot endpoint:

```
https://your-domain.com/api/bot/messages/
```

must be:

- **publicly accessible**
- **HTTPS**
- **stable URL** (not changing every restart)

So if you don't want ngrok, then **yes, you need hosting**.

Simple answer for your case

Local only (`127.0.0.1`)

❌ Teams **cannot** reach:

```
http://127.0.0.1:8000/api/bot/messages/
```

because that is only on your PC.

With ngrok

✅ Teams can reach:

```
https://random-name.ngrok-free.app/api/bot/messages/
```

because ngrok exposes your local PC to the internet.

Without ngrok

✅ You need to deploy your Django app to a **real server/cloud hosting** with HTTPS.

Best options for you (simple + practical)

For your project (**Django + Redis + Celery + Teams bot**), best hosting options are:

1) Render (easiest for beginners) ✅ Recommended

Good for:

- Django web service
- Redis (managed)
- Celery worker
- Celery beat

You can create:

- Web Service
- Redis

- Background Worker (Celery worker)
- Background Worker (Celery beat)

---

2) Railway

Also beginner-friendly.

---

3) DigitalOcean / VPS

More control, but more setup.

---

4) Azure App Service + Azure Cache for Redis

Best if staying in Microsoft ecosystem for Teams bot, but more setup.

For your exact project, I recommend:

✅ Use Render

Because it supports your stack nicely:

- Django app
- Redis
- Celery worker
- Celery beat
- public HTTPS URL automatically

Then your bot endpoint becomes something like:

```
https://wellnessreminder.onrender.com/api/bot/messages/
```

And you can use that in:

- **Azure Bot registration**
- **Teams app manifest / bot endpoint**

Important: If you host it, what runs where?

Instead of your local 4 terminals, hosting will run these as services:

Service 1: Web app

```bash
gunicorn wellnessreminder.wsgi:application
```

Service 2: Celery worker

```bash
celery -A wellnessreminder worker -l info
```

Service 3: Celery beat

```bash
```

```
celery -A wellnessreminder beat -l info
```

Service 4: Redis

- managed Redis (recommended)
- or self-hosted Redis

So the answer in one line

✅ Yes:

If you want to avoid ngrok, you must host your project on a public HTTPS server.

Best path for you right now

Since you already have:

- Django working
- Celery working
- Redis working

Best next move:

Deploy to Render first

Because it will let you:

- skip ngrok
- get stable HTTPS URL
- keep your existing Celery + Redis setup

- test Teams bot properly

My recommendation (strong)

Do this next:

Deploy on Render

Then update:

- `ALLOWED_HOSTS`
- Bot endpoint in Azure
- Teams app config

If you want, I can now give you:

Exact Render deployment steps for your current project

including:

1. `requirements.txt` check
2. `build.sh`
3. `start command`
4. environment variables
5. PostgreSQL or keep SQLite (for testing)
6. Redis setup
7. Celery worker setup
8. Celery beat setup

9. exact bot endpoint to use in Azure
10. final production `settings.py` changes

Reply:

“give me Render deployment step by step for this project”

and I’ll give you the **full exact deployment guide** for your current codebase.